
1 ADJACENCY LABELLING FOR PROPER MINOR-CLOSED GRAPH CLASSES

2 *Anonymous*

3 **ABSTRACT.** We show that every proper minor-closed class of graphs admits a $(1+o(1))\log_2 n$ -
4 bit adjacency labelling scheme. Equivalently, for every proper minor-closed class $\mathcal{G}$ and
5 every positive integer n there exists an $n^{1+o(1)}$ -vertex graph U such that every n -vertex
6 graph in $\mathcal{G}$ is isomorphic to an induced subgraph of U . Both results are optimal up to the
7 lower order term.

8 1 Introduction

9 Let $\mathcal{G}$ be a class of graphs and let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a function. We say that $\mathcal{G}$ admits an *$f(n)$ -bit*
10 *adjacency labelling scheme* if there exists a function $A : (\{0,1\}^*)^2 \rightarrow \{0,1\}$ such that for all
11 positive integers n , for every n -vertex graph $G \in \mathcal{G}$, there exists a function $\ell : V(G) \rightarrow \{0,1\}^*$
12 such that $|\ell(v)| \leq f(n)$ for each vertex v in G , and such that for every two vertices u, v in G ,

$$A(\ell(u), \ell(v)) = \begin{cases} 0 & \text{if } uv \notin E(G), \\ 1 & \text{if } uv \in E(G). \end{cases}$$

14 In this paper we prove the following result. (All logarithms are in base 2.)

15 **Theorem 1.** *Every proper minor-closed class of graphs admits a $(1 + o(1))\log n$ -bit adjacency*
16 *labelling scheme.*

17 Note that all the dependence on the fixed proper minor-closed class in **Theorem 1** is in
18 the $o(\log n)$ term. Also, **Theorem 1** is optimal up to the $o(\log n)$ term, which is $\mathcal{O}((\log n)^{3/4})$.
19 The proof of **Theorem 1** is constructive. For every fixed minor-closed class $\mathcal{G}$, there is a
20 polynomial-time algorithm that takes a graph $G \in \mathcal{G}$ as input and constructs the labelling
21 $\ell : V(G) \rightarrow \{0,1\}^*$.

22 A consequence¹ of **Theorem 1** is the existence of induced-universal graphs with a near-
23 linear number of vertices for n -vertex graphs belonging to a fixed proper minor-closed
24 class of graphs.

25 **Corollary 2.** *For every proper minor-closed class $\mathcal{G}$, for every positive integer n , there exists a*
26 *graph U with $n^{1+o(1)}$ vertices such that every n -vertex graph in $\mathcal{G}$ is isomorphic to an induced*
27 *subgraph of U .*

¹In fact, the viewpoints of adjacency labelling schemes and induced-universal graphs are essentially equivalent. A minor detail is that the labelling schemes need to be injective for the equivalence to hold but this is the case for those developed in this paper. See [28, Section 2.1] for more details on the connection between the two notions.

1.1 State of the Art

Adjacency labeling schemes were introduced in the late 1980s by Kannan, Naor, and Rudich [18] and independently in the PhD thesis of Muller [24]. Since this initial work, adjacency labelling schemes and, more generally, informative labelling schemes have remained a very active area of research. Here we review results most relevant to the current work, namely results on trees, bounded treewidth graphs, and planar graphs.

We start with a simple example to better grasp the notion of labelling schemes. Given an n -vertex tree T , fix an arbitrary vertex to be the root of T , and assign each vertex of T with an identifier, i.e. a number in $\{0, \dots, n-1\}$. For each non-root vertex v of T , let the label of v be the pair consisting of its identifier and the identifier of its parent. For the root vertex of T , let its label be the pair consisting of two copies of its identifier. Clearly, given two labels of vertices in T , a test of adjacency is simply to verify whether an identifier of one vertex is equal to a stored identifier of the parent of the other vertex. This constitutes a $\lceil \log(n^2) \rceil = \lceil 2 \log n \rceil$ -bit adjacency labelling scheme for trees. There is a fascinating series of results improving on this simple scheme for trees. In 1990, Chung [4] gave an adjacency labelling scheme for n -vertex trees with each label of length $\log n + \mathcal{O}(\log \log n)$. Alstrup and Rauhe [2] devised a scheme with labels of length $\log n + \mathcal{O}(\log^* n)$. Finally, in 2015, Alstrup, Dahlgaard, and Knudsen [1] gave a $(\log n + \mathcal{O}(1))$ -bit adjacency labelling scheme for trees, which is optimal up to the $\mathcal{O}(1)$ term. Indeed, consider an n -vertex graph with no two vertices having the same neighborhood (e.g. a path for $n \geq 4$). In any adjacency labelling scheme, the labels of the vertices must be all different and therefore one label must be of length at least $\lceil \log n \rceil$.

In 2007, Gavioille and Labourel [12] presented a $(1 + o(1)) \log n$ -bit adjacency labelling scheme for graphs of bounded treewidth. This is particularly relevant to the topic of this paper because of the following famous application of the Graph Minor Structure Theorem: DeVos, Ding, Oporowski, Sanders, Reed, Seymour, and Vertigan [6] showed that every graph in a proper minor-closed class can be edge 2-coloured so that each monochromatic subgraph has bounded treewidth. Therefore, given a proper minor-closed class of graphs $\mathcal{G}$, for each G in $\mathcal{G}$, fix such a 2-coloring of the edges of G , say with red and blue, and label each vertex of G by the concatenation of the two labels given by the adjacency labelling schemes for the red subgraph of G and for the blue subgraph of G . Thus, all labels are of length $(2 + o(1)) \log n$. Given two labels, to test adjacency simply test whether there is a blue edge or whether there is a red edge. This gives a $(2 + o(1)) \log n$ -bit adjacency labelling scheme for a fixed proper minor-closed class of graphs, which remained the best-known scheme up to the present work.

Planar graphs are a prominent example of a proper minor-closed class of graphs and there is a long history of research on their adjacency labelling schemes. Since planar graphs are 5-degenerate, they admit a simple $\lceil 6 \log n \rceil$ -bit adjacency labelling scheme, which was already observed by Muller [24]. Kannan et al. [18] use a similar approach that makes use of the fact that planar graphs have arboricity 3 (so their edges can be partitioned into three forests [25]) to devise an adjacency labelling scheme for planar graphs whose labels have length at most $\lceil 4 \log n \rceil$. Of course, the later result of Gavioille and Labourel [12] also applies and gives a scheme with labels of length $(2 + o(1)) \log n$. The most recent break-

through came with the application of the product structure theorem for planar graphs by Dujmović, Joret, Micek, Morin, Ueckerdt, and Wood [9]. In 2020, Bonamy, Gavaille, and Pilipczuk [3] recognized that the product structure is particularly useful for adjacency labelling, breaking the $(2 + o(1))\log n$ barrier by introducing a scheme with labels of length $(\frac{4}{3} + o(1))\log n$. Finally, in 2020 Dujmović, Esperet, Gavaille, Joret, Micek, and Morin [8] used the product structure theorem to give a $(1 + o(1))\log n$ -bit adjacency labelling scheme for planar graphs, which is optimal up to the lower order term.

The proof of Dujmović et al. [8] works for graphs embeddable in any fixed surface, and generally for any minor-closed class excluding a fixed apex graph as a minor. Here a graph X is *apex* if X can be made planar by the removal of at most one vertex. Such apex-minor-free classes are the limit of this approach, since a minor-closed class $\mathcal{G}$ has ‘product structure’ if and only if some apex graph is not in $\mathcal{G}$. Indeed, the existence of a $(1 + o(1))\log n$ -bit adjacency labelling scheme for K_t -minor-free graphs is stated as an open problem by Dujmović et al. [8, Section 6, Problem 2]. Prior to the present work, the best result for K_t -minor-free graphs, $t \geq 6$, was the $(2 + o(1))\log n$ -bit adjacency labelling scheme of Gavaille and Labourel [12].

1.2 Proof Overview

We prove [Theorem 1](#) by introducing a new generalization of adjacency labelling, called weighted mixed labelling. It supports vertex weights and, more importantly, also assigns labels to cliques. These labellings are defined for vertex-weighted graphs in which each vertex v is assigned a positive integer weight $\omega(v)$. A *weighted mixed labelling* is defined in terms of a graph G^+ equipped with a weight function ω , a spanning subgraph G of G^+ , and a subset $\mathcal{K}$ of the cliques in G^+ . A mixed labelling of $(G^+, G, \mathcal{K}, \omega)$ is then a function $\mu : V(G^+) \cup \mathcal{K} \rightarrow \{0, 1\}^*$ that assigns a label to each vertex of G^+ and each clique in $\mathcal{K}$. In addition to this, for each clique $K \in \mathcal{K}$, each vertex v in K is assigned a label $\kappa(K, v)$ called the *local identifier* for v in K .

Let $\Omega := \sum_{v \in V(G^+)} \omega(v)$ be the total weight of vertices in G^+ . The efficiency of a mixed labelling is measured by three parameters

- $g_1 = \max_{v \in V(G^+)} (|\mu(v)| - (\log \Omega - \log \omega(v)))$ measures how much the length of a vertex label exceeds the ideal value $\log \Omega - \log \omega(v)$ that appears in Kraft’s Inequality [22].
- $g_2 = \max\{|\kappa(K, v)| : K \in \mathcal{K}, v \in K\}$ is the length of the longest local identifier.
- $g_3 = \max_{K \in \mathcal{K}} (|\mu(K)| - (\log \Omega - \log(\min_{v \in K} \omega(v))))$ measures how much the length of a clique label exceeds the Kraft-like quantity $\log \Omega - \log(\min_{v \in K} \omega(v))$.

Mixed labellings generalize adjacency labellings: Any two vertex labels $\mu(v)$ and $\mu(w)$ are sufficient to perform an adjacency test between the vertices v and w in the subgraph G . In addition to this, for any clique K in $\mathcal{K}$, any vertex v' of G^+ , and any vertex v of K , the labels $\mu(K)$, $\mu(v')$ and $\kappa(K, v)$ are sufficient to test if $v' = v$. The idea here is that $\kappa(K, v)$ provides each vertex v in K a short label that is only useful when $\mu(K)$ is also available. A mixed labelling scheme μ is *efficient* if $g_1, g_2, g_3 \in o(\log n)$. Note that, if we use the uniform weighting $\omega(v) = 1$ for each $v \in V(G)$, then all labels in an efficient mixed labelling have

length $(1 + o(1))\log n$. In particular, such a labelling is a $((1 + o(1))\log n)$ -bit adjacency labelling of G .

Our main technical result, from which [Theorem 1](#) quickly follows, is stated in terms of tree-decompositions. A *tree-decomposition* of a graph G is a pair $(T, (B_x \mid x \in V(T)))$, where T is a tree and $B_x \subseteq V(G)$ for every $x \in V(T)$, with the following properties: (a) for every vertex u in G , the subgraph of T induced by $\{x \in V(T) \mid u \in B_x\}$ is non-empty and connected; and (b) for every edge uv in G , there exists $x \in V(T)$ such that $u, v \in B_x$. We call the sets B_x the *bags* of the tree-decomposition and the sets $B_x \cap B_y$ for all distinct $x, y \in V(T)$, the *adhesions* of the tree-decomposition. The *width* of a tree-decomposition is the maximum size of a bag minus one. The *adhesion-width* of a tree-decomposition is the maximum size of an adhesion.

Let G be a graph, let $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ be a tree-decomposition of G , and let x be a vertex of T . The *torso* of x , denoted $G\langle B_x \rangle$, is the graph with the vertex-set B_x in which two distinct vertices u and v are adjacent if $uv \in E(G)$, or there exists $y \in N_T(x)$ such that $u, v \in B_x \cap B_y$.² A tree-decomposition $(T, (B_x \mid x \in V(T)))$ is *rooted* if T is a rooted tree. The *home node* of a vertex v of G in a rooted tree-decomposition $(T, (B_x \mid x \in V(T)))$ is the minimum-depth node x of T such that $v \in B_x$. The *parent adhesion* of a node x in T is the set of vertices in B_x whose home bag is not x . The *lower torso* at x is the subgraph of $G\langle B_x \rangle$ that is left after removing vertices in the parent adhesion of x . A fundamental property of tree-decompositions is that, for any edge vw of G , the home nodes of v and w are in an ancestor-descendant relationship in T (including the possibility that v and w have the same home node).

Our main technical result can now be stated as:

Theorem 3. *Let $\mathcal{G}$ be a monotone graph class that admits an efficient weighted mixed labelling scheme. Then, for any fixed k , the family $\mathcal{G}'$ of all graphs having tree-decomposition of adhesion-width at most k whose torsos are in $\mathcal{G}$ also admits an efficient weighted mixed labelling scheme.*

The proof of [Theorem 3](#) has a relatively simple high-level description, illustrated in [Figure 1](#). The first step is to partition the vertices of the tree T indexing a tree-decomposition $\mathcal{T}$ into ‘skinny’ subtrees so that contracting each of these subtrees gives a ‘short’ tree T' ([Corollary 9](#)). Here, the skinniness of subtrees and the shortness of T' share a common parameter b where increasing b makes T' shorter (decreases the height of T') but makes the subtrees in the partition less skinny. One can then treat T' as indexing a tree-decomposition $\mathcal{T}'$ of G whose torsos: (a) have skinny tree-decompositions (each defined by one of the subtrees in the partition); and, (b) admit efficient weighted mixed labelling schemes.

The proof then proceeds in two steps: The first step is to show that graphs that have skinny tree-decompositions whose torsos admit efficient weighted mixed labelling schemes also have efficient weighted mixed labelling schemes ([Lemma 14](#)). The second

²For a graph G and a vertex v of G , $N_G(v)$ denotes a set of neighbours of v in G . The notation $G\langle B_x \rangle$ introduces some minor ambiguity since the definition of $G\langle B_x \rangle$ depends on $(B_y \mid y \in N_T(x))$, but the notation makes no reference to the tree-decomposition $(T, (B_z \mid z \in V(T)))$. When we use this notation, the tree-decomposition will always be obvious from context.

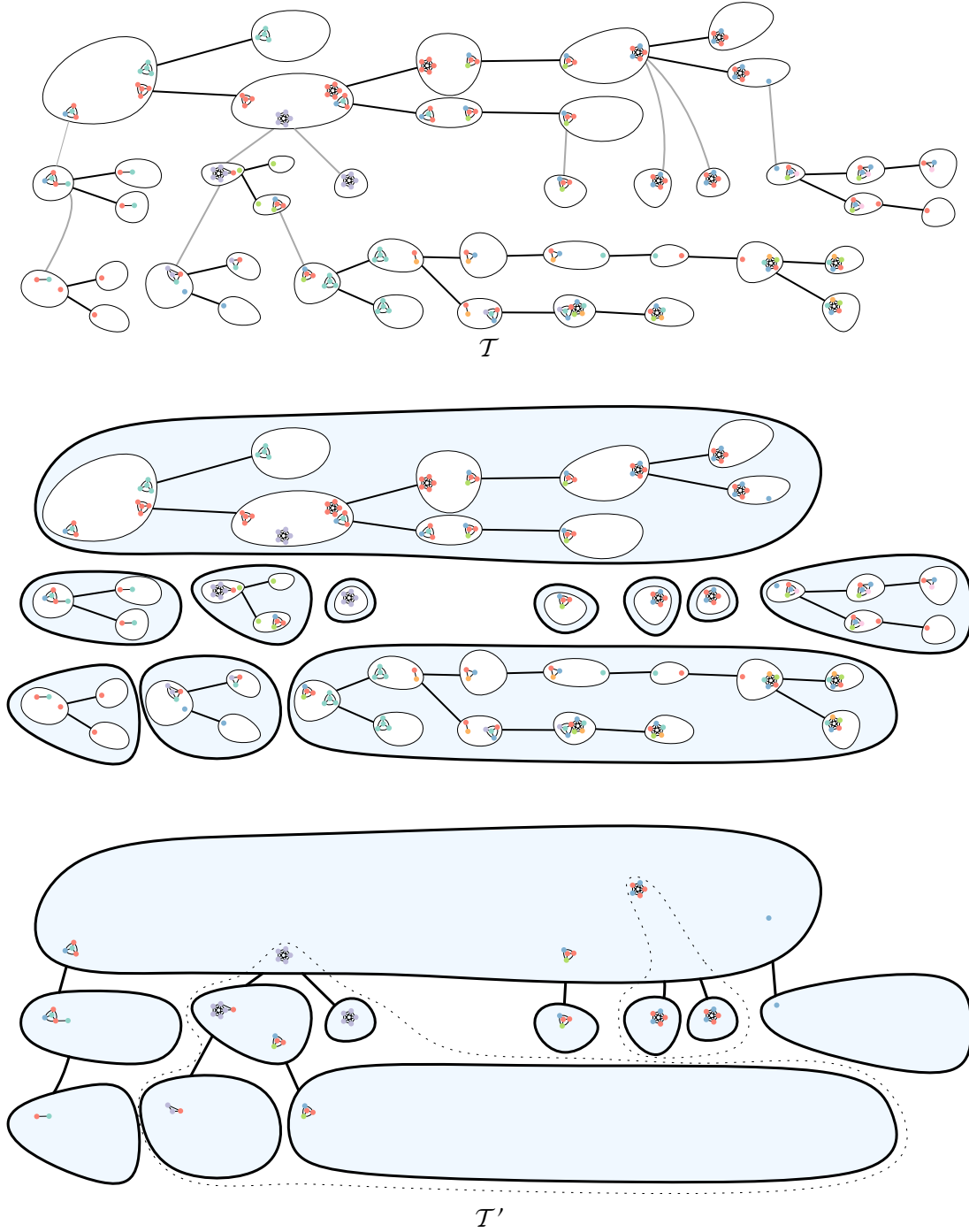


Figure 1: The proof of [Theorem 3](#) works by partitioning the tree T indexing a tree-decomposition $\mathcal{T}$ (top) into skinny subtrees (middle) so that contracting these gives a short tree T' indexing the tree-decomposition $\mathcal{T}'$ (bottom).

shows that graphs that have tree-decompositions of sufficiently small height whose torsos admit efficient weighted mixed labelling schemes also admit efficient weighted mixed labelling schemes (Lemma 15). These are then combined to give a more precise version of Theorem 3 that quantifies the efficiency of the resulting weighted mixed labelling scheme (Lemma 16).

The first step makes use of the fact that each skinny subtree of T defines a skinny tree-decomposition which we linearize to obtain a path decomposition with adhesion-width $\mathcal{O}(bk)$. The solution of this path-decomposition-based subproblem involves a combination of techniques that have been developed for similar (though unweighted) problems [8, 12]. The extension to the weighted version (and mixed labellings) is achieved by using alphabetic binary trees rather than the balanced binary trees used in [8, 12]. One critical detail that our construction ensures and the solution makes use of, is that it guarantees more than just the adhesion-width bound $\mathcal{O}(bk)$. In the linearization we use the fact that each vertex has at most k neighbours in the parent adhesion of its home node, and this is important.

The generalization provided by weighted mixed labelling schemes is crucial for the second step, as we now explain. Let w be a vertex of G and let $x_0, \dots, x_s$ be the path from the root x_0 of the short tree T' to the home node x_s of w . (Most of) the vertex label $\mu(w)$ is obtained by concatenating the labels $\mu_{x_0}(K_1), \dots, \mu_{x_{s-1}}(K_s), \mu_{x_s}(w)$ where μ_{x_0} is a mixed labelling of $G\langle B_{x_0} \rangle$, μ_{x_i} is a mixed labelling of $G\langle B_{x_i} \rangle - B_{x_{i-1}}$, and K_i is a clique whose vertex-set is contained in the adhesion $B_{x_{i-1}} \cap B_{x_i}$, for each $i \in \{1, \dots, s\}$. This serves two purposes:

1. For two vertices v and w , the longest common prefix of $\mu(v)$ and $\mu(w)$ can be used to determine if the home nodes in T' for v and w are in an ancestor-descendant relationship (or if v and w have the same home node.)
2. The vertex label $\mu(w)$ for w can be augmented to explicitly store $\kappa(K_{x_i}, v)$ for each neighbour v of w whose home node is a strict ancestor x_i of w 's home node x_s . Each such vertex v is necessarily contained in the adhesion $B_{x_s} \cap B_{x_{s-1}}$. Since the adhesion-width of T' is at most k , this adds at most $k \cdot g_2(n)$ to the length of $\mu(w)$.

The total length of vertex labels is controlled by using the appropriate weight function ω_x when computing the weighted mixed labelling for the lower torso of each node x of T' .

Despite the simplicity of this high-level description, getting all of this to work involves carefully managing the interplay between the skinniness parameter b , the input parameters g_1 , g_2 , and g_3 of the weighted mixed labelling scheme for each torso of the original tree-decomposition $\mathcal{T}$, the parameters g'_1 , g'_2 and g'_3 of the weighted mixed labelling scheme for graphs with skinny tree-decompositions, and the parameters g''_1 , g''_2 and g''_3 of the weighted mixed labelling scheme for the entire graph. For example, the lengths of the labels $\mu_{x_0}(K_1), \dots, \mu_{x_{s-1}}(K_s), \mu_{x_s}(w)$ described above form a telescoping series that sums to $\log \Omega - \log \omega(w) + s \cdot g'_3 + g'_1$, so the value of g''_1 is at least $s \cdot g'_3 + g'_1$, which is why we require that T' is short, since the height of T' upper bounds s . This requires using a large value of b , which makes the subtrees used to define T' less skinny, this increases the value of g'_2 , which increases the length of the second part of $\mu(w)$, which has length

$k \cdot g'_2(n)$. Optimizing the trade-off between the skinniness parameter b and the height h of T' ultimately yields the $O((\log n)^{3/4})$ lower order term in our final bound.

To deduce [Theorem 1](#) from [Theorem 3](#), we use two ingredients. The first ingredient is a rough version of the Robertson-Seymour Graph Minor Structure Theorem due to Dujmović et al. [9] which states that every graph in a proper-minor-closed family $\mathcal{G}$ has a tree-decomposition whose torsos have ‘product-plus-apex structure’. More specifically, each torso is a graph that, after the removal of at most $a(\mathcal{G})$ vertices, is a subgraph of a strong graph product $H \boxtimes P$, where P is a path and the treewidth of H is at most $t(\mathcal{G})$. The second ingredient is a generalization of the adjacency labelling scheme for ‘product structured’ graphs of Dujmović et al. [8] to the weighted mixed labelling setting. This generalization, which is discussed in [Appendix A](#), is fairly straightforward: The notion of clique labels and membership labels is already implicit in [8] and the addition of vertex weights is accomplished by a standard trick of simulating weight by multiplicity.

1.3 Paper organization

The remainder of the paper is organized as follows: [Section 2](#) reviews some basic background material and establishes the relationship between the skinniness of subtrees and the height of T' discussed above. [Section 3](#) introduces weighted mixed labelling schemes and proves [Theorem 3](#). [Section 4](#) describes the Graph Minor Product Structure Theorem from [9], the adaptation of the adjacency labelling scheme from [8] to the setting of weighted mixed labellings, and puts everything together to prove [Theorem 1](#). We conclude in [Section 5](#) by explaining how the constructive nature of [Theorem 1](#) leads to a polynomial-time labelling algorithm.

2 Building blocks

Let $\mathbb{N}$ be the set of all nonnegative integers, and let $\mathbb{N}^+$ be the set of all positive integers.

In this paper, all graphs are finite, simple, and undirected. For a graph G , $V(G)$ denotes the vertex-set of G , $E(G)$ denotes the edge-set of G , and $K(G)$ denotes the set of cliques in G , where a *clique* is a set of pairwise adjacent vertices. For standard graph theory terms, we refer the reader to the textbook by Diestel [7].

2.1 Trees

Let T be a rooted tree and let r be the root of T . For each $x \in V(T)$, define $P_T(x)$ to be the path in T from x to r and the *T -depth* of x , denoted $\text{depth}_T(x)$, to be the number of edges in $P_T(x)$. The *height* of T , denoted $\text{height}(T)$, is the maximum T -depth of a vertex in T . For each $y \in V(T)$ and each $x \in V(P_T(y))$ (including y), the vertex x is a T -ancestor of y and y is a *T -descendant* of x . For each $x \in V(T)$, define T_x to be the subtree of T induced by all T -descendants of x . For each edge xy of T where x is a T -ancestor of y , we say that x is the *T -parent* of y and y is a *T -child* of x . We say that a set $S \subseteq V(T)$ is a *T -antichain*, if there are no distinct $x, y \in S$ such that x is a T -ancestor of y . For any set $S \subseteq V(T)$, the *lowest common T -ancestor* of S is the node in $\bigcap_{x \in S} V(P_T(x))$ of maximum T -depth. (When there is no danger of ambiguity, we may drop the leading T - from T -depth, T -ancestor, T -descendant, T -parent, and T -child.) For each $i \in \mathbb{N}^+$, the *i -th layer* of T is

$L_i(T) := \{x \in V(T) : \text{depth}_T(x) = i - 1\}$. Thus, the root of T is the only vertex in L_1 .

2.2 Tree-Decompositions

A tree-decomposition $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ is *normalised* if $B_x \not\subseteq B_y$ for every two distinct nodes $x, y \in V(T)$. Any tree-decomposition $\mathcal{T}$ can be normalised through a sequence of edge contractions and bag mergers to produce a normalised tree-decomposition $\mathcal{T}'$ whose width and adhesion-width are not greater than the width and adhesion-width of $\mathcal{T}$, respectively. Furthermore, every torso of $\mathcal{T}'$ is also a torso of $\mathcal{T}$. A rooted tree-decomposition $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ is *downward-normalised* if the root r of T has a non-empty bag B_r and, for each node y of T and each T -ancestor $x \neq y$ of y , $B_y \not\subseteq B_x$. Clearly, if $\mathcal{T}$ is normalised then it is also downward-normalised for any choice of root $r \in V(T)$. A rooted tree-decomposition $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ is *upmost* if, for each adhesion A of $\mathcal{T}$, there exists a node x of T such that, for any edge yz of T with $B_y \cap B_z \subseteq A$, $x \in \{z, y\}$ and the node in $\{z, y\} \setminus \{x\}$ is a T -child of x . Any downward-normalised rooted tree-decomposition $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ of a graph G can be made upmost by swapping edges in T to obtain a tree T' such that $\mathcal{T}' := (T', (B_x \mid x \in V(T')))$ is a tree-decomposition of G , each adhesion in $\mathcal{T}'$ is an adhesion in $\mathcal{T}$ and each torso in $\mathcal{T}'$ is a subgraph of a torso in $\mathcal{T}$.

We will often take subgraphs of a graph G defined by subtrees of a tree indexing a tree-decomposition of G . When we do this, we will (implicitly) make use of the following fact: If a rooted tree-decomposition $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ is downward-normalised and upmost then, for any node x of T and any T -child y of x , the tree-decomposition $\mathcal{T}_y := (T_y, (B_z \setminus B_x \mid z \in V(T_y)))$ is a downward-normalised upmost tree-decomposition of the graph $G_y := G[\bigcup_{z \in V(T_y)} B_z] - B_x$. Furthermore, $B_v \cap B_w \neq \emptyset$ for each edge vw of T , then $B_v \cap B_w \setminus B_z \neq \emptyset$ for each edge vw of T_y . In words, if every edge of T defines a non-empty adhesion in $\mathcal{T}$, then every edge of T_y defines a non-empty adhesion in $\mathcal{T}_y$. Throughout the rest of this paper, we will implicitly assume that any rooted tree-decomposition we are considering is downward-normalised and upmost.

Let G be a graph and $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ be a tree-decomposition of G . If T is a path, then $\mathcal{T}$ is a *path-decomposition* of G , simply denoted by the sequence $(W_1, W_2, \dots, W_m)$ of the bags in the order of the vertices in the underlying path.

2.3 Weight Functions

For a non-empty set S , a *weight function* over S is a mapping $\omega : S \rightarrow \mathbb{R}^+$ of the elements of S onto positive real numbers. When X is a subset of S , we use the shorthand $\omega(X) := \sum_{x \in X} \omega(x)$. When $S := V(G)$ is the vertex-set of a graph G and H is a subgraph of G , we use the shorthand $\omega(H) := \omega(V(H))$.

Let $\omega : S \rightarrow \mathbb{R}^+$ be a weight function. For each $x \in S$, the *(Kraft) ideal codeword length* for x is $\log \omega(S) - \log \omega(x)$. It will be convenient to place some upper bound on the Kraft ideal codeword length or, equivalently, on the ratio of total weight, $\omega(S)$ to minimum weight $\min_{x \in S} (\omega(x))$. The following observation gives a way to do this without increasing the ideal codeword length of each $x \in S$ by more than 1:

Observation 4. *For every weight function $\omega : S \rightarrow \mathbb{R}^+$, there exists a weight function $\omega'(S) \rightarrow \mathbb{N}^+$ such that $\log \omega'(S) - \log \omega'(x) \leq \min\{\log |S|, \log \omega(S) - \log \omega(x)\} + 2$, for each $x \in S$.*

Proof. By dividing every weight by $\min_{x \in S} \omega(x)$ we may assume, without loss of generality, that $\omega(x) \geq 1$ for each $x \in S$, since this does not change the value of the right-hand side in the inequality. For each $x \in S$, define $\omega'(x) := \lceil \max\{\omega(S)/|S|, \omega(x)\} \rceil$. Then $\omega'(S) < \sum_{x \in S} (\omega(S)/|S| + \omega(x) + 1) = 2\omega(S) + |S| \leq 3\omega(S)$. For each $x \in S$, the inequality $\omega'(x) \geq \omega(x)$ implies that $\log \omega'(S) - \log \omega'(x) \leq \log \omega(S) - \log \omega(x) + \log 3$ and the inequality $\omega'(x) \geq \omega(S)/|S|$ implies that $\log \omega'(S) - \log \omega'(x) \leq \log \omega(S) - \log(\omega(S)/|S|) + \log 3 = \log |S| + \log 3$. $\square$

Normalised rooted tree-decompositions define a useful weight function over the nodes of their indexing trees. Let G be a graph and let $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ be a non-redundant rooted tree-decomposition. We define a weight function $\omega_{\mathcal{T}} : V(T) \rightarrow \mathbb{N}^+$ as follows. For each node x of T , we define

$$\omega_{\mathcal{T}}(x) = \begin{cases} |B_x| & \text{if } x \text{ is the root of } T, \\ |B_x \setminus B_y| & \text{if } x \text{ is not the root of } T \text{ and } y \text{ is the } T\text{-parent of } x. \end{cases}$$

Note that $\sum_{x \in V(T)} \omega_{\mathcal{T}}(x) = |V(G)|$.

2.4 Shannon and Shannon–Fano–Elias Codes

A *code* for a set S is an injective function $\lambda : S \rightarrow \{0, 1\}^*$ that maps each element in S to a distinct binary string. A code is *prefix-free* if $\lambda(x)$ is not a prefix of $\lambda(y)$ for all distinct $x, y \in S$. We make use of the following fundamental result by Shannon.

Lemma 5 ([26, 27]). *Let S be a non-empty set and let $\omega : S \rightarrow \mathbb{R}^+$. Then there exists a prefix-free code $\pi : S \rightarrow \{0, 1\}^*$ such that $|\pi(x)| \leq \log \omega(S) - \log \omega(x) + 1$ for each $x \in S$.*

A rooted tree T is a *binary tree* if each node of T has at most two children among which at most one is the *left child* and at most one is the *right child*. A binary tree T is *full* if for every node x of T , either x has two children or x has no children. A full binary tree T is *complete* if all the leaves of T have the same depth in T . Let T be a binary tree. There is a natural ordering, called *left-to-right ordering*, of the leaves of T , such that for every node x , if x has two children then all the leaf descendants of the left child of x precede all the leaf descendants of the right child of x .

The following result is a variant of Alphabetic Binary Trees. Similar results (with the constant 3 replaced by 2) are well-known and there are several proofs [14, 21, 23]. We provide a proof of the weaker statement here because it illustrates a simple but powerful technique—*simulating weight by multiplicity*—that we will use again later.

Lemma 6. *Let S be a linearly ordered set and let $\omega : S \rightarrow \mathbb{N}^+$. Then there exists a binary tree T such that S is the set of leaves of T , the linear order on S agrees with the left-to-right-order of the leaves in T , and for each $x \in S$*

$$\text{depth}_T(x) \leq \log \omega(S) - \log \omega(x) + 3.$$

Proof. Let $h := \lceil \log \omega(S) \rceil$. Let Q be the complete binary tree of height h , so with 2^h leaves. For each $x \in S$, define the $\omega(x)$ -element set

$$S_{\omega}(x) := \{(x, 1), (x, 2), \dots, (x, \omega(x))\}$$

and let $S_\omega := \bigcup_{x \in S} S_\omega(x)$. Then $|S_\omega| = \omega(S)$. Treat S_ω as a linearly ordered set by using lexicographic order and rename the leftmost $\omega(S)$ leaves of Q , in order, with elements of S_ω .

Now for each $x \in S$, we are going to fix a node $r(x)$ in Q such that (1) all the leaves of $Q_{r(x)}$ lie in $S_\omega(x)$; and (2) $Q_{r(x)}$ contains at least $\omega(x)/4$ leaves.

Let $x \in S$. If $\omega(x) = 1$ then define $r(x) := (x, 1)$, which is a leaf of Q . Clearly, (1) and (2) holds. Now suppose that $\omega(x) \geq 2$. Consider the lowest common Q -ancestor y of $S_\omega(x)$. Thus, y is an internal node of Q and both children of y contain in their subtree a leaf in $S_\omega(x)$. Since $S_\omega(x)$ is a set of consecutive leaves in Q , all leaves of the left child y' of y in Q that are in $S_\omega(x)$ are rightmost in $Q_{y'}$, and all leaves of the right child y'' of y in Q that are in $S_\omega(x)$ are leftmost in $Q_{y''}$. Let z be the child of y in Q such that Q_z has at least $\omega(x)/2$ leaves in $S_\omega(x)$ and let S_z be the set of all such leaves. Thus, $|S_z| \geq \omega(x)/2$. Since the argument is completely symmetric in these cases, assume that z is the left child of y . Consider a path in Q_z starting from z and always taking an edge to the right child of the current node until we finish at a leaf. Let v be the first vertex of that path such that Q_v has all leaves in S_z . If $v = z$ then we set $r(x) := v$. Clearly (1) and (2) holds. Suppose that $v \neq z$. Let u be the parent of v . Since the set of leaves of Q_u form an interval of rightmost leaves of Q_z and since Q_u contains a leaf not in S_z , we conclude that Q_u contains all nodes in S_z as leaves. Therefore, Q_v contains at least $|S_z|/2$ vertices of S_z as leaves. We set $r(x) := v$ and again (1) and (2) holds.

Note that the set $\{r(x) \mid x \in S\}$ is a Q -antichain. Indeed, if $x, y \in S$, $x \neq y$ and $r(x)$ is a Q -ancestor of $r(y)$, then the leaves of $Q_{r(y)}$ are contained in the leaves of $Q_{r(x)}$ which by (1) implies that $Q_{r(x)}$ contains all nodes in $S_\omega(y)$ as leaves. However, again by (1) all the leaves of $Q_{r(x)}$ lie in $S_\omega(x)$ which is disjoint from $S_\omega(y)$, a contradiction.

Let $x \in S$. Since $Q_{r(x)}$ contains at least $\omega(x)/4$ leaves, we have that $\text{height}(T_{r(x)}) \geq \log(\omega(x)/4) = \log \omega(x) - 2$. Finally,

$$\begin{aligned} \text{depth}_Q(r(x)) &\leq \text{height}(Q) - \text{height}(Q_{r(x)}) \\ &\leq \lceil \log \omega(S) \rceil - (\log \omega(x) - 2) \\ &\leq \log \omega(S) - \log \omega(x) + 3. \end{aligned}$$

Let $T := \bigcup_{x \in S} P_Q(r(x))$. Since $\{r(x) \mid x \in S\}$ is a Q -antichain, we conclude that $\{r(x) \mid x \in S\}$ is the set of all leaves of T . Identifying each leaf r_x of T with x for each $x \in S$ gives the desired tree. $\square$

In coding theory, [Lemma 6](#) is the basis of the Shannon-Fano-Elias coding scheme, as we now explain. By assigning each edge e of a binary tree T a 0 or 1 depending on whether e joins a parent to its left or right child, we obtain an encoding of each root-to-leaf path in T as a binary string. Doing this with the tree from [Lemma 6](#), we obtain the following.

Corollary 7. *Let S be a linearly ordered set and let $\omega : S \rightarrow \mathbb{N}^+$. Then there exists a code $\rho : S \rightarrow \{0, 1\}^*$ for S such that:*

- (a) $|\rho(x)| \leq \log \omega(S) - \log \omega(x) + 3$ for each $x \in S$, and
- (b) $\rho(x)$ is lexicographically less than $\rho(y)$ for each $x, y \in S$ with $x < y$ in S .

2.5 On Multipart Labels

Throughout the remainder of the paper, we will describe labelling schemes in which the labels (bitstrings) have multiple parts, of varying lengths. We write this as $s := (s_1, s_2, \dots, s_p)$, where each of $s_1, \dots, s_p$ is a (possibly empty) bitstring, so s has length $|s| = \sum_{i=1}^p |s_i|$. This is, of course, not completely correct because for a decoder to extract the individual parts $s_1, \dots, s_p$, it must know the number of parts p and at least $p-1$ of the lengths $|s_1|, \dots, |s_p|$. We handle this by encoding the value of p and the lengths $|s_1|, \dots, |s_p|$ and prepending these to s . This is most easily done using the Elias γ encoding of positive integers, in which each $x \in \mathbb{N}^+$ is encoded by a self-delimiting binary string $\gamma(x)$ with $|\gamma(x)| \leq \lfloor 2 \log x \rfloor + 1$ [11].³ Thus, the “true” encoding of s is $(\gamma(p), \gamma(|s_1| + 1), \dots, \gamma(|s_p| + 1), s_1, \dots, s_p)$.

In our setting it will always be the case that $p \in o(\log n)$ and we will use [Observation 4](#) to ensure that $|s_i| \in \mathcal{O}(\log n)$ for each $i \in [p]$. Thus, these extra bookkeeping bits increase the length of the encoding of s by $\mathcal{O}(p \log \log n)$. To avoid a series of pointless distractions, we will not discuss this extra information when describing our labels, but the calculation of label lengths will include extra terms of the form $\mathcal{O}(p \log \log n)$ that account this extra bookkeeping.

2.6 Separators with Low Pathwidth and Low Path-Adhesion-Width

For an integer $b \in \mathbb{N}^+$, a rooted tree T is *b -skinny*, if $|L_i(T)| \leq b$ for each $i \in \mathbb{N}^+$. A rooted tree-decomposition $(T, (B_x \mid x \in V(T)))$ of a graph G is *b -skinny* if T is b -skinny. The following lemma shows how to construct a $(1/b)$ -separator using the bags in a b -skinny subtree of a tree-decomposition.

Lemma 8. *Let $b > 1$, let T be a rooted tree, let $\omega : V(T) \rightarrow \mathbb{R}^+$, and let*

$$X := \{x \in V(T) \mid \omega(T_x) > \omega(T)/b\}.$$

Then:

- (a) $T[X]$ is connected and contains the root of T ;
- (b) $|X \cap L_i(T)| < b$, for each $i \in \mathbb{N}^+$; and
- (c) $\omega(C) \leq \omega(T)/b$ for each component C of $T - X$.

Proof. Let r be the root of T . Since $b > 1$, we have $\omega(T_r) = \omega(T) > \omega(T)/b$. For $x \in X \setminus \{r\}$, let y be the T -parent of x , then $\omega(T_y) \geq \omega(y) + \omega(T_x) > \omega(T_x) > \omega(T)/b$. Therefore $r \in X$ and, for each $x \in X \setminus \{r\}$, X contains the T -parent of x . Thus $T[X]$ is connected. This proves (a).

For each $i \in \mathbb{N}^+$, the subtrees in $\{T_x \mid x \in X \cap L_i(T)\}$ are pairwise vertex-disjoint, so

$$\omega(T) \geq \sum_{x \in X \cap L_i(T)} \omega(T_x) > \sum_{x \in X \cap L_i(T)} \omega(T)/b = |X \cap L_i(T)| \cdot \omega(T)/b.$$

Rewriting this inequality gives $|X \cap L_i(T)| < b$ for each $i \in \mathbb{N}^+$. This proves (b).

By (a), each component C of $T - X$ is a subtree of T that is rooted at some node y whose T -parent is in X . Since $y \notin X$, $\omega(C) = \omega(T_y) \leq \omega(T)/b$. This proves (c). $\square$

³For each $x \in \mathbb{N}^+$, $\gamma(x)$ consists of $\lfloor \log x \rfloor$ 0 bits, followed by the binary representation of x (which begins with a 1 bit and has length $\lfloor \log x \rfloor + 1$).

Corollary 9. For every $b > 1$, every $n \in \mathbb{N}^+$, every n -vertex graph G and every tree-decomposition $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ of G , there exists a tree-decomposition $\mathcal{Q} := (Q, (D_x \mid x \in V(Q)))$ of G such that

- (a) each adhesion of $\mathcal{Q}$ is an adhesion of $\mathcal{T}$;
- (b) for each $q \in V(Q)$, $G\langle D_q \rangle$ has a b -skinny tree-decomposition $(T(q), (B_y \mid y \in V(T(q))))$ where $T(q)$ is a subtree of T ; and
- (c) for each $q \in V(Q)$, $\omega_{\mathcal{Q}}(Q_q) \leq n/b^{\text{depth}_{\mathcal{Q}}(q)}$.

Proof. Fix $b > 1$. The proof is an induction on n . To make the induction work we require additionally that

- (d) $B_r \subseteq D_s$ where s is the root of Q .

Let G be an n -vertex graph, let $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ be a rooted tree-decomposition of G , and let r be the root of T .

If $n \leq b$, let Q be a 1-node tree and let s be the only node which is also the root. Let $D_s := \bigcup_{y \in X} B_y$ and let $\mathcal{Q} := (Q, (D_q)_{q \in V(Q)})$. Then $\mathcal{Q}$ is a tree-decomposition with no adhesions, so (a) holds vacuously. Since $G\langle D_s \rangle = G$, we can take $T(s) = T$ and then $(T(s), (B_y \mid y \in V(T(s))))$ is a b -skinny tree decomposition of $G\langle D_s \rangle$, so (b) holds. Since $V(Q) = \{s\}$ and $\text{depth}_{\mathcal{Q}}(s) = 0$, (c) holds. Also by construction (d) holds.

Now assume $n > b$. Let $X := \{x \in V(T) \mid \omega_{\mathcal{T}}(T_x) > n/b\}$. By Lemma 8, $T[X]$ is a b -skinny subtree of T that includes r . By Lemma 8, for each $y \in N_T(X)$ the subtree T_y has weight $\omega_{\mathcal{T}}(T_y) = |V(G_y)| \leq n/b$. For each $y \in N_T(X)$, apply the inductive hypothesis to subgraph $G_y := G[\bigcup_{z \in V(T_y)} B_z]$ with tree-decomposition $\mathcal{T}_y := (T_y, (B_x \mid x \in V(T_y)))$ to obtain a tree-decomposition $\mathcal{Q}_y := (Q_y, (D_x \mid x \in V(Q_y)))$ whose root bag contains B_y . Construct a tree Q as follows. Start with a single vertex s that will be the root of Q and set $D_s := \bigcup_{x \in X} B_x$. For each $y \in N_T(X)$, attach Q_y to Q by adding an edge joining the root of Q_y to s . This gives the desired tree Q and the desired tree-decomposition $\mathcal{Q} = (Q, (D_y \mid y \in V(Q)))$.

For each $y \in N_T(X)$, each adhesion of $\mathcal{T}_y$ is an adhesion of $\mathcal{T}$. Thus, $\mathcal{Q}$ satisfies (a) because each adhesion A of $\mathcal{Q}$ is either an adhesion of $\mathcal{Q}_y$ or a $A = D_s \cap D_q$ for some Q -child q of s . In the former case, A is an adhesion of $\mathcal{T}_y$, by the inductive hypothesis. In the latter case $A = B_x \cap B_y$ for some $x \in X$ and some $y \in N_T(x)$.

All that remains is to show that each $q \in V(Q)$ satisfies (c). It is clear that $q = s$ satisfies (c) since $\text{depth}_{\mathcal{Q}}(s) = 0$. Now consider some $q \in V(Q) \setminus \{s\}$. Let t be the Q -ancestor of q with $\text{depth}_{\mathcal{Q}}(t) = 1$ (so t is a Q -child of s). By Lemma 8, $\omega_{\mathcal{Q}}(Q_t) \leq n/b$. By the inductive hypothesis:

$$\omega_{\mathcal{Q}}(Q_q) \leq \omega_{\mathcal{Q}}(Q_t)/b^{\text{depth}_{\mathcal{Q}_t}(q)} \leq n/b^{\text{depth}_{\mathcal{Q}_t}(q)+1} = n/b^{\text{depth}_{\mathcal{Q}}(q)} . \quad \square$$

3 Weighted Mixed Labelling Schemes – Proof of Theorem 3

We now describe the generalization of adjacency labelling schemes that we use to prove Theorem 3. In the following, one can think of the graph G^+ as a torso $H\langle B_x \rangle$ in a tree-decomposition $\mathcal{T}$ of a larger graph H , the graph G as an induced subgraph $H[B_x]$, and the cliques in $\mathcal{K}$ as adhesions in $\mathcal{T}$.

Let $\mathcal{G}$ be a class of graphs. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ for each $i \in [3]$. The class $\mathcal{G}$ admits a *weighted (g_1, g_2, g_3) mixed labelling scheme* of $\mathcal{G}$ if there exists a pair (A, M) with

- $A : (\{0, 1\}^*)^2 \rightarrow \{0, 1\}$ (the *adjacency tester*); and
- $M : (\{0, 1\}^*)^3 \rightarrow \{0, 1\}$ (the *identity tester*)

such that for every natural $n \in \mathbb{N}$, every n -vertex graph G^+ in $\mathcal{G}$, every spanning subgraph G of G^+ , every set of cliques $\mathcal{K}$ in $K(G^+)$, and every weight function $\omega : V(G) \rightarrow \mathbb{R}^+$, there exist $\mu : V(G) \cup \mathcal{K} \rightarrow \{0, 1\}^*$ and $\kappa : \{(K, u) \mid K \in \mathcal{K}, u \in K\} \rightarrow \{0, 1\}^*$ such that

(i) for all $u, v \in V(G)$

$$A(\mu(u), \mu(v)) = 1 \text{ if and only if } uv \in E(G);$$

(ii) for all $K \in \mathcal{K}, u \in K, v \in V(G)$

$$M(\mu(K), \kappa(K, u), \mu(v)) = 1 \text{ if and only if } u = v;$$

(iii) for every $v \in V(G)$

$$|\mu(v)| \leq \log \omega(G) - \log \omega(v) + g_1(n);$$

(iv) for every $K \in \mathcal{K}$

$$|\mu(K)| \leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3(n);$$

(v) for every $K \in \mathcal{K}$ and $u \in K$

$$|\kappa(K, u)| \leq g_2(n).$$

Given μ and κ as above we say that μ is the *mixed labelling* of $(G^+, G, \mathcal{K}, \omega)$ for (A, M) and κ is the *local identifier* of $(G^+, G, \mathcal{K}, \omega)$ for (A, M) .

In the proofs, it will be convenient to consider a slightly weaker notion of weighted (g_1, g_2, g_3) mixed labelling schemes. A weight function $\omega : S \rightarrow \mathbb{N}^+$ is *nice* if

$$\log \omega(S) - \log \omega(x) \leq \log |S| + 2$$

holds for every $x \in S$. (Let us emphasize that weights are required to be positive integers here, which will be important later on.) We then say that a class of graphs $\mathcal{G}$ admits a *weak weighted (g_1, g_2, g_3) mixed labelling scheme* if $\mathcal{G}$ satisfies the definition given above for weighted (g_1, g_2, g_3) mixed labelling schemes, with the exception that only nice weight functions ω are considered. The two notions are essentially equivalent, as shown by the following lemma. This will allow us to only consider nice weight functions in the proofs.

Lemma 10. *Let $\mathcal{G}$ be a class of graphs. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ be functions for each $i \in [3]$. If $\mathcal{G}$ admits a weak weighted (g_1, g_2, g_3) mixed labelling scheme, then $\mathcal{G}$ admits a weighted $(g_1 + 2, g_2, g_3 + 2)$ mixed labelling scheme.*

Proof. Suppose that $\mathcal{G}$ admits a weak weighted (g_1, g_2, g_3) mixed labelling scheme. Let A and M be the corresponding adjacency and identity tester, respectively. We will use the same pair (A, M) for our weighted $(g_1 + 2, g_2, g_3 + 2)$ mixed labelling scheme. Let $n \in \mathbb{N}$, let G^+ be a n -vertex graph in $\mathcal{G}$, let G be a spanning subgraph G , let $\mathcal{K}$ be a set of cliques

in $K(G^+)$, and let $\omega : V(G) \rightarrow \mathbb{R}^+$ be an arbitrary weight function. Let ω' be the function obtained from [Observation 4](#). Thus,

$$\log \omega'(V(G)) - \log \omega'(v) \leq \min\{\log n, \log \omega(G) - \log \omega(v)\} + 2$$

holds for every $v \in V(G)$. In particular, ω' is nice. Hence, by our assumption, there is a mixed labelling μ of $(G^+, G, \mathcal{K}, \omega')$ for (A, M) and a local identifier κ of $(G^+, G, \mathcal{K}, \omega')$ for (A, M) . Hence,

$$|\mu(v)| \leq \log \omega'(G) - \log \omega'(v) + g_1(n) \leq \log \omega(G) - \log \omega(v) + g_1(n) + 2$$

holds for every $v \in V(G)$, and

$$|\mu(K)| \leq \log \omega'(G) - \log \min_{v \in K}(\omega'(v)) + g_3(n) \leq \log \omega(G) - \log \min_{v \in K}(\omega(v)) + g_3(n) + 2.$$

holds for every $K \in \mathcal{K}$. It follows that μ and κ are the desired mixed labelling and local identifier, respectively, of $(G^+, G, \mathcal{K}, \omega)$ for (A, M) . $\square$

The existence of a weighted $(g_1(n), \cdot, \cdot)$ mixed labelling scheme for a graph class $\mathcal{G}$ immediately implies the existence of a $(\log n + g_1(n))$ -bit adjacency labelling scheme for $\mathcal{G}$. Thus, we are interested in the case where $g_1(n) \in o(\log n)$. We say that a graph family $\mathcal{G}$ admits an *efficient* weighted mixed labelling scheme if $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme with $g_i(n) \in o(\log n)$ for each $i \in [3]$.

The advantage of weighted mixed labelling schemes over adjacency labelling schemes is, roughly, that efficient weighted mixed labelling schemes are closed under taking tree-decompositions of bounded adhesion-width. More precisely, if $\mathcal{G}$ is a monotone graph class that admits an efficient weighted mixed labelling scheme then, for any fixed k , the family of graphs that have tree-decompositions of adhesion-width at most k whose torsos are in $\mathcal{G}$ also admits an efficient weighted mixed labelling scheme. This is [Theorem 3](#) in Subsection 1.2, which the rest of this section is devoted to proving.

3.1 Vertex-Disjoint Unions of Graphs

Lemma 11. *Let $\mathcal{G}$ be a class of graphs and let $\mathcal{G}'$ be the class of graphs that can be formed by taking the union of a finite number of pairwise vertex-disjoint graphs in $\mathcal{G}$. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ be a non-decreasing function with $g_i(n) \in \mathcal{O}(\log n)$ for each $i \in [3]$. Suppose that $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme. Then $\mathcal{G}'$ admits a weighted (g'_1, g_2, g'_3) mixed labelling scheme, where*

$$\begin{aligned} g'_1(n) &= g_1(n) + \mathcal{O}(\log \log n) , \\ g'_3(n) &= g_3(n) + \mathcal{O}(\log \log n) . \end{aligned}$$

Proof. Let A and M denote the adjacency and identity testers, respectively, for the weighted (g_1, g_2, g_3) mixed labelling scheme of $\mathcal{G}$.

By [Lemma 10](#), we only need to show that $\mathcal{G}'$ admits a weak weighted (g'_1, g_2, g'_3) mixed labelling scheme. Let $n \in \mathbb{N}$, let G^+ be an n -vertex graph in $\mathcal{G}'$, let G be a spanning subgraph

of G^+ , let $\mathcal{K}$ be a set of cliques in G^+ , and let $\omega : V(G) \rightarrow \mathbb{N}^+$ be a nice weight function. Thus, for each $v \in V(G)$ we have

$$\log \omega(G) - \log \omega(v) \leq \log n + 2. \quad (1)$$

By definition, $G^+ := \bigcup_{i=1}^m G_i^+$, where $G_1^+, \dots, G_m^+$ are pairwise vertex-disjoint members of $\mathcal{G}$. Therefore, $G := \bigcup_{i=1}^m G_i$, where $G_i := G[V(G_i^+)]$ is a spanning subgraph of G_i^+ for each $i \in [m]$. For each $i \in [m]$, let $\mathcal{K}_i := \{K \in \mathcal{K} \mid K \subseteq V(G_i^+)\}$. Note that $\{\mathcal{K}_i \mid i \in [m]\}$ is a partition of $\mathcal{K}$.

The subgraph label $\pi(x)$: Let $\psi : [m] \rightarrow \mathbb{N}^+$ be the weight function defined by $\psi(i) := \omega(G_i)$ and observe that $\psi([m]) = \omega(G)$ since $G_1, \dots, G_m$ are pairwise vertex-disjoint. Apply [Lemma 5](#) with $S := [m]$ and weight function ψ to obtain a prefix-free code $\pi : [m] \rightarrow \{0, 1\}^*$ such that for each $i \in [m]$ we have

$$|\pi(i)| \leq \log \omega(G) - \log \omega(G_i) + 1. \quad (2)$$

For each $i \in [m]$ and each $x \in V(G_i) \cup \mathcal{K}_i$, define $\pi(x) := \pi(i)$.

The sub-label $\mu_i(x)$: For each $i \in [m]$, define weight function $\omega_i : V(G_i) \rightarrow \mathbb{N}^+$ by restricting $\omega : V(G) \rightarrow \mathbb{N}^+$ to the vertices of G_i . For each $i \in [m]$, let $\mu_i : V(G_i) \cup \mathcal{K}_i \rightarrow \{0, 1\}^*$ be the mixed labelling of $(G_i^+, G_i, \mathcal{K}_i, \omega_i)$ for (A, M) . Thus, for each $i \in [m]$ and each $v \in V(G_i)$ we have

$$\begin{aligned} |\mu_i(v)| &\leq \log \omega(G_i) - \log \omega(v) + g_1(|V(G_i)|) \\ &\leq \log \omega(G_i) - \log \omega(v) + g_1(n) \end{aligned} \quad (3)$$

and for each $K \in \mathcal{K}_i$ we have

$$\begin{aligned} |\mu_i(K)| &\leq \log \omega(G_i) - \log \min_{v \in K} (\omega(v)) + g_3(|V(G_i)|) \\ &\leq \log \omega(G_i) - \log \min_{v \in K} (\omega(v)) + g_3(n). \end{aligned} \quad (4)$$

The vertex label $\mu(v)$: For each $i \in [m]$ and each $v \in V(G_i)$, define $\mu(v) := (\pi(v), \mu_i(v))$ so we have

$$\begin{aligned} |\pi(v)| + |\mu_i(v)| &\leq \log \omega(G) - \log \omega(v) + 1 + g_1(n) && \text{by (2) and (3),} \\ &\leq \log n + 3 + g_1(n) = \mathcal{O}(\log n) && \text{by (1),} \end{aligned}$$

and therefore

$$\begin{aligned} |\mu(v)| &\leq |\pi(v)| + |\mu_i(v)| + \mathcal{O}(\log(|\pi(v)| + |\mu_i(v)|)) \\ &\leq \log \omega(G) - \log \omega(v) + g_1(n) + \mathcal{O}(\log \log n). \end{aligned}$$

Adjacency testing: We define an adjacency tester A' as follows. Given the labels $\mu(v)$ and $\mu(w)$ for two vertices $v, w \in V(G)$, we compute $A'(\mu(v), \mu(w))$ as follows. First we verify if $\pi(v) = \pi(w)$. If $\pi(v) \neq \pi(w)$, then v and w lie in different components of G , so they cannot be adjacent in G , and we set $A'(\mu(v), \mu(w)) = 0$. If $\pi(v) = \pi(w)$, then let $i = \pi^{-1}(v)$. Thus in this case both v and w lie in G_i , $\mu(v) = (\pi(i), \mu_i(v))$, and $\mu(w) = (\pi(i), \mu_i(w))$. In this case, we simply set $A'(\mu(v), \mu(w)) = A(\mu_i(v), \mu_i(w))$.

The clique label $\mu(K)$: For each $i \in [m]$ and each $K \in \mathcal{K}_i$, define $\mu(K) := (\pi(K), \mu_i(K))$, so we have

$$\begin{aligned} |\pi(K)| + |\mu_i(K)| &\leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + 1 + g_3(n) && \text{by (2) and (4),} \\ &\leq \log n + 3 + g_3(n) = \mathcal{O}(\log n) && \text{by (1),} \end{aligned}$$

and therefore

$$\begin{aligned} |\mu(K)| &\leq |\pi(K)| + |\mu_i(K)| + \mathcal{O}(\log(|\pi(K)| + |\mu_i(K)|)) \\ &\leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3(n) + \mathcal{O}(\log \log n). \end{aligned}$$

The local identifier $\kappa(K, u)$: For each $i \in [m]$, let $\kappa_i : \{(K, u) \mid K \in \mathcal{K}_i, u \in K\} \rightarrow \{0, 1\}^*$ be the local identifier of $(G_i^+, G_i, \mathcal{K}_i, \omega_i)$ for (A, M) . We define $\kappa(K, u) = \kappa_i(K, u)$, for each $i \in [m]$, each $K \in \mathcal{K}_i$, and each $u \in K$. In particular, we have

$$|\kappa(K, u)| \leq g_2(n).$$

Identity testing: We define an identity tester M' as follows. Given $\mu(K)$, $\mu(v)$ and $\kappa(K, u)$ for some $K \in \mathcal{K}$, some $v \in V(G)$ and some $u \in K$, we compute $M'(\mu(K), \kappa(K, u), \mu(v))$ as follows. First we verify if $\pi(K) = \pi(v)$. If $\pi(K) \neq \pi(v)$, then K and v lies in different components of G , so $v \notin K$, and we set $M'(\mu(K), \kappa(K, u), \mu(v)) = 0$. If $\pi(K) = \pi(v)$, then let $i = \pi^{-1}(K)$. Thus in this case both K and v lie in G_i , so we have $\mu(K) = (\pi(i), \mu_i(K))$, $\mu(v) = (\pi(i), \mu_i(v))$, and $\kappa(K, u) = \kappa_i(K, u)$. In this case, we simply set $M'(\mu(K), \kappa(K, u), \mu(v)) = M(\mu(K), \kappa(K, u), \mu(v))$. $\square$

For technical reasons, we need a variant of [Lemma 11](#), in which the sub-labellings $\mu_1, \dots, \mu_m$ are already given, as in the following lemma. The proof of [Lemma 11](#) immediately extends to this case.

Lemma 12. *Let $\mathcal{G}$ be a class of graphs and let $\mathcal{G}'$ be the class of graphs that can be formed by taking the union of a finite number of pairwise vertex-disjoint graphs in $\mathcal{G}$. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ be a non-decreasing function with $g_i(n) \in \mathcal{O}(\log n)$ for each $i \in [3]$. Suppose that $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme, and let (A, M) be a corresponding pair of adjacency and identity testers. Then $\mathcal{G}'$ admits a weighted (g'_1, g_2, g'_3) mixed labelling scheme, where*

$$\begin{aligned} g'_1(n) &= g_1(n) + \mathcal{O}(\log \log n) , \\ g'_3(n) &= g_3(n) + \mathcal{O}(\log \log n) . \end{aligned}$$

Furthermore, there is such a labelling scheme with a corresponding pair (A', M') of adjacency and identity testers satisfying the following extra property: Let $m \in \mathbb{N}^+$, for each $i \in [m]$, let G_i^+ be a graph, let G_i be a spanning subgraph of G_i^+ , let $\mathcal{K}_i \subseteq \mathcal{K}(G_i^+)$, let $\omega_i : V(G_i) \rightarrow \mathbb{R}^+$, and let $\mu_i : V(G_i) \cup \mathcal{K}_i \rightarrow \{0, 1\}^*$ be a mixed labelling of $(G_i^+, G_i, \mathcal{K}_i, \omega_i)$ for (A, M) . Suppose further that the graphs G_i^+ ($i \in [m]$) are pairwise vertex-disjoint, and let $G^+ := \bigcup_{i=1}^m G_i^+$, $G := \bigcup_{i=1}^m G_i$, $\mathcal{K} := \bigcup_{i=1}^m \mathcal{K}_i$, and let $\omega(v) := \omega_i(v)$ for each $i \in [m]$ and each $v \in V(G_i^+)$. Then there is a mixed labelling $\mu : V(G) \cup \mathcal{K} \rightarrow \{0, 1\}^*$ of $(G^+, G, \mathcal{K}, \omega)$ for (A', M') such that

$$|\mu(x)| \leq \log \omega(G) - \log \omega(G_i) + |\mu_i(x)| + \mathcal{O}(\log \log n),$$

for each $i \in [m]$ and each $x \in V(G_i^+) \cup \mathcal{K}_i$.

3.2 Adding Apex Vertices

The *complete join* of two graphs G_1 and G_2 , denoted by $G_1 \oplus G_2$, is the graph obtained from the disjoint union of G_1 and G_2 by adding all edges vw with $v \in V(G_1)$ and $w \in V(G_2)$. For a graph class $\mathcal{G}$ and a graph J , let $\mathcal{G} \oplus J := \{G \oplus J \mid G \in \mathcal{G}\}$.

Lemma 13. *Let $\mathcal{G}$ be a class of graphs. Let $a \in \mathbb{N}^+$. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ be a non-decreasing function with $g_i(n) \in \mathcal{O}(\log n)$ for each $i \in [3]$. Suppose that $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme. Then $\mathcal{G} \oplus K_a$ admits a weighted (g'_1, g'_2, g'_3) mixed labelling scheme, where*

$$\begin{aligned} g'_1(n) &= g_1(n) + \mathcal{O}(a + \log \log n) , \\ g'_2(n) &= g_2(n) + \mathcal{O}(\log a) , \\ g'_3(n) &= g_3(n) + \mathcal{O}(a + \log \log n) . \end{aligned}$$

Proof. Let A and M denote the adjacency and identity testers, respectively, for the weighted (g_1, g_2, g_3) mixed labelling scheme of $\mathcal{G}$.

By Lemma 10, it is enough to show that $\mathcal{G} \oplus K_a$ admits a weak weighted (g'_1, g'_2, g'_3) mixed labelling scheme. Let $n \in \mathbb{N}$, let G^+ be an n -vertex graph in $\mathcal{G} \oplus K_a$, let G be a spanning subgraph of G^+ , let $\mathcal{K}$ be a set of cliques in G^+ , and let $\omega : V(G) \rightarrow \mathbb{N}^+$ be a nice weight function. Let $B := V(K_a) \cap V(G^+)$. Then $G^+ - B$ and $G - B$ are each in $\mathcal{G}$. Let $\mathcal{K}^- := \{K - B \mid K \in \mathcal{K}, K \not\subseteq V(B)\}$. Define the weight function $\omega^- : V(G - B) \rightarrow \mathbb{N}^+$ by restricting $\omega : V(G) \rightarrow \mathbb{N}^+$ to the vertices of $G - B$.

The subgraph label μ^- : Let $\mu^- : V(G - B) \cup \mathcal{K}^- \rightarrow \{0, 1\}^*$ be the mixed labelling of $(G^+ - B, G - B, \mathcal{K}^-, \omega^-)$ for (A, M) . Then for each $v \in V(G - B)$ we have

$$\begin{aligned} |\mu^-(v)| &\leq \log \omega(G - B) - \log \omega(v) + g_1(|V(G - B)|) \\ &\leq \log \omega(G) - \log \omega(v) + g_1(n). \end{aligned} \tag{5}$$

and for each $K \in \mathcal{K}^-$ we have

$$\begin{aligned} |\mu^-(K)| &\leq \log \omega(G - B) - \log \min_{v \in K} (\omega(v)) + g_3(|V(G - B)|) \\ &\leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3(n). \end{aligned} \tag{6}$$

Extend the domain of μ^- to include the vertices in B by defining $\mu^-(v) := \varepsilon$ (the empty string), for each $v \in B$.

The apex identifier $\sigma(u)$: Fix a linear ordering $u_1, \dots, u_b$ of the vertices in B . For each $u \in V(G)$ define

$$\sigma(u) = \begin{cases} \lceil \log(a+1) \rceil\text{-bit binary representation of } i & \text{if } u \in B \text{ and } u = u_i, \\ \lceil \log(a+1) \rceil \text{ zero bits} & \text{if } u \in V(G) - B. \end{cases}$$

The vertex label $\mu(v)$: For each $v \in V(G)$ and each $i \in [b]$ let $c_i(v)$ be a bit that indicates if $vu_i \in E(G)$. For each $v \in V(G)$, define $\mu(v) := (\sigma(v), c_1(v), \dots, c_b(v), \mu^-(v))$. Then for each $v \in V(G)$ we have

$$|\sigma(v)| + b + |\mu^-(v)| \leq \lceil \log(a+1) \rceil + a + \log \omega(G) - \log \omega(v) + g_1(n)$$

$$\begin{aligned}
&\leq \log \omega(G) - \log \omega(v) + g_1(n) + 2a \\
&\leq \log n + g_1(n) + 2a + 2 && \text{since } \omega \text{ is nice} \\
&= \mathcal{O}(a + \log n),
\end{aligned}$$

and therefore

$$\begin{aligned}
|\mu(v)| &\leq |\sigma(v)| + b + |\mu^-(v)| + \mathcal{O}(\log(|\sigma(v)| + b + |\mu^-(v)|)) \\
&\leq \log \omega(G) - \log \omega(v) + g_1(n) + \mathcal{O}(a + \log \log n).
\end{aligned}$$

Adjacency testing: We define an adjacency tester A' as follows. Given the labels $\mu(v)$ and $\mu(w)$ for two vertices v and w of G , we compute $A'(\mu(v), \mu(w))$ as follows. First by checking the values of $\sigma(v)$ and $\sigma(w)$ we check if v and/or w lie in B . If at least one of them, say v , is in B then the tester knows the value $i \in [b]$ such that $v = u_i$ and therefore we can set $A'(\mu(v), \mu(w)) = c_i(w)$. If neither v nor w is in B , then we set $A'(\mu(v), \mu(w)) = A(\mu^-(v), \mu^-(w))$.

The clique label $\mu(K)$: First, extend the domain of μ^- so that it includes the empty set, by setting $\mu^-(\emptyset) := \varepsilon$ (the empty string). For each $K \in \mathcal{K}$ and each $i \in [b]$, let $c_i(K)$ be a bit that indicates if $u_i \in K$. For each $K \in \mathcal{K}$, define $\mu(K) := (\mu^-(K - B), c_1(K), \dots, c_b(K))$. Then

$$\begin{aligned}
|\mu^-(K - B)| + b &\leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3(n) + a && \text{by (6)} \\
&\leq \log n + 2 + g_3(n) + a && \text{since } \omega \text{ is nice} \\
&= \mathcal{O}(a + \log n),
\end{aligned}$$

and therefore

$$\begin{aligned}
|\mu(K)| &\leq |\mu^-(K - B)| + b + \mathcal{O}(\log(|\mu^-(K - B)| + b)) \\
&\leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3(n) + \mathcal{O}(a + \log \log n).
\end{aligned}$$

The local identifier $\kappa(K, u)$: Let κ^- be the local identifier of $(G^+ - B, G - B, \mathcal{K}^-, \omega^-)$ for (A, M) . For each clique $K \in \mathcal{K}$ and each $u \in K$, we define

$$\kappa(K, u) := \begin{cases} (0, \kappa^-(K - B, u)) & \text{if } u \notin B, \\ (1, \sigma(u)) & \text{if } u \in B. \end{cases}$$

In particular, we have

$$|\kappa(K, u)| \leq 1 + \max\{g_2(n), \lceil \log(a + 1) \rceil\} \leq g_2(n) + \mathcal{O}(\log a).$$

Identity testing: We define an identity tester M' as follows. Given $\mu(K)$, $\kappa(K, u)$, $\mu(v)$ for some $K \in \mathcal{K}$, $u \in K$, and $v \in V(G)$, we compute $M'(\mu(K), \kappa(K, u), \mu(v))$ as follows. First by checking the value of $\mu(v)$ and the first bit of $\kappa(K, u)$, the tester determines whether $v \in B$ and whether $u \in B$. If $u, v \in B$, then we set $M'(\mu(K), \kappa(K, u), \mu(v)) = 1$ if $\kappa(K, u) = (1, \sigma(v))$, and $M'(\mu(K), \kappa(K, u), \mu(v)) = 0$ otherwise. If $u, v \notin B$, then we set $M'(\mu(K), \kappa(K, u), \mu(v)) = M(\mu^-(K - B), \kappa^-(K, u), \mu^-(v))$. Otherwise, one of the vertices is in B and the other is not, so we set $M'(\mu(K), \kappa(K, u), \mu(v)) = 0$. $\square$

3.3 Graphs with Skinny Tree-Decompositions

Lemma 14. Let $\mathcal{G}$ be a monotone class of graphs. Let $b, k \in \mathbb{N}^+$. Let $\mathcal{G}'$ be the class of graphs consisting of every graph that has a b -skinny tree-decomposition of adhesion-width at most k in which each torso is a member of $\mathcal{G}$. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ be a non-decreasing function with $g_i(n) \in \mathcal{O}(\log n)$ for each $i \in [3]$. Suppose that $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme. Then $\mathcal{G}'$ admits a weighted (g'_1, g'_2, g'_3) mixed labelling scheme, where

$$\begin{aligned} g'_1(n) &= g_1(n) + \mathcal{O}(k \log \log n + k \log(bk)), \\ g'_2(n) &= \max\{g_2(n) + \mathcal{O}(1), \mathcal{O}(\log(bk) + \log \log n)\}, \\ g'_3(n) &= g_3(n) + \mathcal{O}(\log \log n). \end{aligned}$$

Proof. Let $\mathcal{G}''$ be the class of graphs obtained by taking the union of a finite number of pairwise vertex-disjoint graphs in $\mathcal{G}$. Applying Lemma 11, we fix A'' and M'' , the adjacency and identity testers, respectively, for a weighted (g''_1, g''_2, g''_3) mixed labelling scheme of $\mathcal{G}''$, where $g''_1(n) = g_1(n) + \mathcal{O}(\log \log n)$ and $g''_3(n) = g_3(n) + \mathcal{O}(\log \log n)$.

As before, by Lemma 10 it is enough to show that $\mathcal{G}'$ admits a weak weighted (g'_1, g'_2, g'_3) mixed labelling scheme. Let G^+ be an n -vertex graph in $\mathcal{G}'$, let G be a spanning subgraph of G^+ , let $\mathcal{K}$ be a set of cliques in G^+ , and let $\omega : V(G) \rightarrow \mathbb{N}^+$ be a nice weight function. By definition, G^+ has a b -skinny tree-decomposition $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ in which each torso belongs to $\mathcal{G}$ and each adhesion has size at most k . Let $p = \text{height}(T) + 1$. For each $i \in [p]$, let $B_i := \bigcup_{x \in L_i(T)} B_x$. Then $\mathcal{P} := (B_1, \dots, B_p)$ is a path-decomposition of G in which each adhesion has size at most bk . Let $C_1 := \emptyset$ and, for each $i \in \{2, \dots, p\}$, let $C_i := B_i \cap B_{i-1}$. Let $C := \bigcup_{i \in [p]} C_i$ be the set of vertices of G that participate in adhesions.

The layer label $\rho(v)$: Define the weight function $\psi : [p] \rightarrow \mathbb{N}^+$ by $\psi(i) := \omega(B_i \setminus C_i)$. Since $\{B_i \setminus C_i \mid i \in [p]\}$ is a partition of $V(G)$, we have $\psi([p]) = \sum_{i \in [p]} \psi(i) = \omega(G)$. Apply Lemma 6 and Corollary 7 to the set $S := [p]$ with weight function ψ , and let T_ψ and $\rho : S \rightarrow \{0, 1\}^*$ be the resulting binary tree and code, respectively. For each $i \in [p]$ and each $v \in B_i \setminus C_i$, define $\rho(v) := \rho(i)$, so we have

$$|\rho(v)| \leq \log \psi(S) - \log \psi(i) + 3 = \log \omega(G) - \log \psi(i) + 3. \quad (7)$$

Also

$$\begin{aligned} \text{height}(T_\psi) &\leq \log \omega(G) - \log \psi(i) + 3 \\ &= \mathcal{O}(\log n). \end{aligned} \quad (8)$$

The mixed sub-label $\mu'(v)$: For each $i \in [p]$, let $G_i^+ := \bigcup_{x \in L_i(T)} G^+[B_x] - C_i$, let $G_i := \bigcup_{x \in L_i(T)} G[B_x] - C_i$, and let $\mathcal{K}_i := \{K \cap B_i \setminus C_i \mid K \in \mathcal{K}\} \setminus \{\emptyset\}$. Note that for each $i \in [p]$ and each $x \in L_i(T)$, $G[B_x] \in \mathcal{G}$ (by definition), and also since $\mathcal{G}$ is monotone we have $G[B_x] - C_i \in \mathcal{G}$. Therefore, for each $i \in [p]$, $G_i^+ \in \mathcal{G}''$. For each $i \in [p]$, let $\omega_i : V(G_i) \rightarrow \mathbb{N}^+$ be the weight function obtained by restricting ω to the vertices of G_i and observe that $\omega_i(G_i) = \psi(i)$. For each $i \in [p]$ let $\mu_i : V(G_i^+) \cup \mathcal{K}_i \rightarrow \{0, 1\}^*$ and $\kappa_i : \{(K, u) \mid K \in \mathcal{K}_i, u \in K\} \rightarrow \{0, 1\}^*$ be the mixed labelling and local identifier of $(G_i^+, G_i, \mathcal{K}_i, \omega_i)$ for (A'', M'') .

For all $i \in [p]$, $v \in V(G_i)$, $K \in \mathcal{K}_i$ we have

$$|\mu_i(v)| \leq \log \psi(i) - \log \omega(v) + g_1''(n), \quad (9)$$

$$|\mu_i(K)| \leq \log \psi(i) - \log \min_{v \in K}(\omega(v)) + g_3''(n), \text{ and} \quad (10)$$

for each $u \in K$,

$$|\kappa_i(K, u)| \leq g_2(n). \quad (11)$$

For each $i \in [p]$ and each $v \in B_i \setminus C_i$, let $\mu'(v) := \mu_i(v)$. Similarly, for each $i \in [p]$, and each $K \in \mathcal{K}_i$, let $\rho(K) := \rho(i)$ and let $\mu'(K) := \mu_i(K)$,

The adhesion identifier $\kappa(v) = (d(v), \varphi(v))$: For each vertex v in G , let

$$a(v) = \min\{i \in [p] \mid v \in B_i\},$$

$$b(v) = \max\{i \in [p] \mid v \in B_i\},$$

$$\text{lca}(v) = \text{lca}(T_\psi, \{a(v), b(v)\}),$$

$$d(v) = \text{depth}_{T_\psi}(\text{lca}(v)).$$

Let x be a node in T_ψ . We define $L_x := \{v \in C \mid \text{lca}(v) = x\}$. We claim that $|L_x| \leq bk$. In order to prove that consider a vertex $v \in C$ such that $\text{lca}(v) = x$. Since $v \in C$, we have $a(v) < b(v)$ so $\text{lca}(v) = x$ must have two children in T_ψ . Consider the largest leaf i in the subtree of the left child of x in T_ψ and the smallest leaf j in the subtree of the right child of x in T_ψ . Then, $j = i + 1$, and $a(v) \leq i < i + 1 \leq b(v)$. Therefore, $v \in B_i \cap B_{i+1}$. We conclude that $|L_x| \leq |B_i \cap B_{i+1}| \leq bk$, as desired.

For each node x in T_ψ , we define $\varphi_x : L_x \rightarrow [bk]$ to be an arbitrary injective function. (It exists because $|L_x| \leq bk$.) Now, we define $\varphi : C \rightarrow [bk]$ so that $\varphi(v) := \varphi_{\text{lca}(v)}(v)$ for each $v \in C$.

Furthermore, for each $v \in V(G)$, let

$$\kappa(v) = \begin{cases} (d(v), \varphi(v)) & \text{if } v \in C, \\ \varepsilon & \text{if } v \notin C. \end{cases}$$

Clearly, we have

$$\begin{aligned} |d(v)| + |\varphi(v)| &\leq \lceil \log(\text{height}(T_\psi) + 1) \rceil + \lceil \log(bk) \rceil \\ &= \mathcal{O}(\log \log n + \log(bk)), \end{aligned}$$

and

$$\begin{aligned} |\kappa(v)| &\leq |d(v)| + |\varphi(v)| + \log(|d(v)| + |\varphi(v)|) \\ &= \mathcal{O}(\log \log n + \log(bk)). \end{aligned}$$

Finally, for each $i \in [p]$ and each $w \in B_i \setminus B_{i-1}$, let $\alpha(w) := (\kappa(v) \mid v \in N_G(w) \cap C_i)$. Note that $|N_G(w) \cap C_i| \leq k$; indeed, if $w \in B_x$ with $x \in V(T)$ and y is the parent of x in T , then $N_G(w) \cap C_i \subseteq B_x \cap B_y$, and $|B_x \cap B_y| \leq k$ since T has adhesions of size at most k . Therefore,

$$|\alpha(w)| \leq \mathcal{O}(k \log \log n + k \log(bk)). \quad (12)$$

The vertex label $\mu(v)$: For each $v \in V(G)$, we define

$$\mu(v) := (\rho(v), \mu'(v), \alpha(v), \kappa(v)).$$

For each $v \in V(G)$, we have

$$\begin{aligned} |\rho(v)| + |\mu'(v)| &\leq \log \omega(G) - \log \omega(v) + g_1''(n) + 3 && \text{by (7) and (9)} \\ &\in \mathcal{O}(\log n), && \text{since } \omega \text{ is nice and } g_1'' \in \mathcal{O}(\log n). \end{aligned}$$

Therefore, for each $v \in V(G)$

$$\begin{aligned} |\rho(v)| + |\mu'(v)| + |\alpha(v)| + |\kappa(v)| &\leq \log \omega(G) - \log \omega(v) + g_1(n) + \mathcal{O}(k \log \log n + k \log(bk)) \\ &= \mathcal{O}(\log n + k \log \log n + k \log(bk)) \end{aligned}$$

and

$$\begin{aligned} |\mu(v)| &\leq |\rho(v)| + |\mu'(v)| + |\alpha(v)| + |\kappa(v)| + \mathcal{O}(\log(|\rho(v)| + |\mu'(v)| + |\alpha(v)| + |\kappa(v)|)) \\ &\leq \log \omega(G) - \log \omega(v) + g_1(n) + \mathcal{O}(k \log \log n + k \log(bk)). \end{aligned}$$

Adjacency Testing: Given the labels $\mu(v)$ and $\mu(w)$ for two vertices v and w of G , the adjacency tester A' works as follows. First, the tester compares $\rho(v)$ and $\rho(w)$:

- If $\rho(v) = \rho(w)$ then v and w are both vertices of G_i , for some $i \in [p]$ so $vw \in E(G)$ if and only if $vw \in E(G_i)$. In this case we set $A'(\mu(v), \mu(w)) = A''(\mu'(v), \mu'(w)) = A''(\mu_i(v), \mu_i(w))$.
- If $\rho(v) \neq \rho(w)$ then assume without loss of generality that $\rho(v)$ is lexicographically less than $\rho(w)$. In this case $v \in B_i \setminus C_i$ and $w \in B_j \setminus C_j$ for some $i < j$. The edge vw can only be present in G if $v \in C_j$. First the tester checks whether $\kappa(v)$ is not empty to see if $v \in C$. If $v \notin C$, then $A'(\mu(v), \mu(w)) = 0$.

Otherwise, the tester checks if $\rho(v)$ and $\rho(w)$ have the same prefix of length $d(v)$. If not then the interval of v in the path-decomposition $(B_1, \dots, B_p)$ is disjoint from the interval of w and therefore these vertices cannot be adjacent, so we set $A''(\mu(v), \mu(w)) = 0$.

Suppose that $\rho(v)$ and $\rho(w)$ have the same prefix of length $d(v)$. In this case the tester inspects each item of $\alpha(w)$. Let (d_0, ϕ_0) be such an item. Note that if this item comes from a $u_0 \in N_G(w) \cap C_j$ then $\text{lca}(u_0)$ must lie on the path in T_ψ from the root to $\rho(w)$. Thus, if $(d_0, \phi_0) = (d(v), \phi(v))$ then the tester concludes that the corresponding neighbour u_0 of w is mapped by $\text{lca}(\cdot)$ to $\text{lca}(v)$ and $\phi(u_0) = \phi(v)$, which implies that $u_0 = v$. In this case, we set $A'(\mu(v), \mu(w)) = 1$. Finally, if no item (d_0, ϕ_0) in $\alpha(w)$ satisfies $(d_0, \phi_0) = (d(v), \phi(v))$, then v and w cannot be adjacent in G , and we set $A'(\mu(v), \mu(w)) = 0$.

The clique labels $\mu(K)$ and local identifiers $\kappa(K, v)$: Fix some $K \in \mathcal{K}$ and let $i := \min\{i : K \cap B_i \neq \emptyset\}$. By the minimality of i , $K \cap B_i \setminus C_i = K \cap B_i$. Let $j := \max\{i : K \cap B_i \setminus C_i \neq \emptyset\}$. Then j is well-defined and $i \leq j$, since $K \cap B_i \setminus C_i \neq \emptyset$. By the choice of j , $K - C_j$ is non-empty, so $K - C_j \in \mathcal{K}_j$. Therefore $K - C_j$ is assigned a label $\mu_j(K - C_j)$ in the mixed labelling of $(G_j^+, G_j, \mathcal{K}_j, \omega_j)$ for (A, M) . Let $\rho(K) := \rho(j)$ and define the clique label

$$\mu(K) := (\rho(K), \mu'(K \setminus C_j)) .$$

Then

$$|\rho(K)| + |\mu'(K \setminus C_j)| \leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3''(n) + 3 \quad \text{by (7) and (10)} \quad (13)$$

$$\in \mathcal{O}(\log n) \quad (14)$$

since ω is nice and $g_3''(n) \in \mathcal{O}(\log n)$. Thus

$$\begin{aligned} |\mu(K)| &\leq |\rho(K)| + |\mu'(K \setminus C_j)| + \mathcal{O}(\log |\rho(K)| + \log |\mu'(K \setminus C_j)|) \\ &\leq \log \omega(G) - \log \min_{v \in K \setminus C_j} (\omega(v)) + g_3''(n) + \mathcal{O}(\log \log n) \quad \text{by (13) and (14)} \\ &\leq \log \omega(G) - \log \min_{v \in K} (\omega(v)) + g_3(n) + \mathcal{O}(\log \log n) . \end{aligned}$$

Recall that the local identifier $\kappa_j(K \setminus C_j, v)$ is defined for each $v \in K \setminus C_j$. For each $v \in K$, we define

$$\kappa(K, v) := \begin{cases} (0, \kappa_j(K \setminus C_j, v)) & \text{if } v \in B_j \setminus C_j \\ (1, \kappa(v)) & \text{if } v \in C_j. \end{cases}$$

In the first case, $|\kappa(K, v)| \leq g_2(n) + \mathcal{O}(1)$. In the second case $|\kappa(K, v)| \in \mathcal{O}(\log(bk) + \log \log n)$.

Identity Testing: Given the labels $\mu(K)$, $\kappa(K, u)$, and $\mu(v)$ for some $K \in \mathcal{K}$, and some $u \in K$, and some $v \in V(G)$ the identity tester M' first compares $\rho(K)$ and $\rho(v)$.

- If $\rho(K)$ is lexicographically smaller than $\rho(v)$, then $K \subseteq \bigcup_{i=1}^j B_i$ and $v \in B_{j'} \setminus C_{j'}$ for some $j < j'$. The definition of $\rho(K)$ implies that $K \cap B_{j'} \setminus C_{j'} = \emptyset$, so $v \neq u$, since $u \in K$. In this case $M'(\mu(K), \kappa(K, u), \mu(v)) := 0$.
- If $\rho(K) = \rho(v)$ then $K \setminus C_j \in \mathcal{K}_j$, and $v \in B_j \setminus C_j$ for some $j \in [p]$. In this case, v can only be equal to u if $v \in B_j \setminus C_j$ and $u \in K \setminus C_j$. The identity tester checks if $u \in K \setminus C_j$ by looking at the first bit of $\kappa(K, u)$. If this bit is 1, then $u \notin K \setminus C_j$, so $M'(\mu(K), \kappa(K, u), \mu(v)) := 0$. Otherwise, $u \in K \setminus C_j$ and $\kappa(K, u)$ contains $\kappa_j(K \setminus C_j, u)$. In this case, $K \setminus C_j$ is a clique in G_j^+ and u and v are vertices of G_j^+ , so $M'(\mu(K), \kappa(K, u), \mu(v)) := M''(\mu_j(K \setminus C_j), \kappa_j(K \setminus C_j, u), \mu_j(v))$. Evaluating M'' on these inputs is possible because $\mu_j(K - C_j)$ is included in $\mu(K)$, $\mu'(v) = \mu_j(v)$ is included in $\mu(v)$, and $\kappa_j(K \setminus C_j, u)$ is included in $\kappa(K, u)$.
- If $\rho(K)$ is lexicographically larger than $\rho(v)$ then $K \setminus C_j \in \mathcal{K}_j$ and $v \in B_i \setminus C_i$ for some $i < j$. The tester examines the first bit of $\kappa(K, u)$ to determine if $u \in K \setminus C_j$. If so, then $u \in B_j \setminus C_j$ and $v \in B_i \setminus C_i$ so $u \neq v$ and $M'(\mu(K), \kappa(K, u), \mu(v)) := 0$. Otherwise,

$\kappa(K, u) = (1, \kappa(u))$. Since $u \in B_j$ and $u \notin K \setminus C_j \subseteq B_j \setminus C_j$, $a(u) < j \leq b(u)$. In particular, $u \in C_j$, so $\text{lca}(u)$ is a T_ψ -ancestor of j . Therefore, $\rho(K)[1, \dots, d(u)] = \rho(j)[1, \dots, d(u)] = \rho(u)[1, \dots, d(u)]$. Therefore, the identity tester checks if $\rho(K)[1, \dots, d(u)] = \rho(v)[1, \dots, d(u)]$. If not, then $\rho(u)[1, \dots, d(u)] \neq \rho(v)[1, \dots, d(u)]$ so $v \neq u$ and $M'(\mu(K), \kappa(K, u), \mu(v)) := 0$.

If the tester is not done by now, then it has deduced that $\text{lca}(u)$ is a T_ψ -ancestor of i . Next the tester checks if $\kappa(v) = \epsilon$. If so, then $v \notin C$ and $u \in C$, so $u \neq v$. Otherwise, $\kappa(v) = (d(v), \varphi(v))$ and the tester checks if $d(u) = d(v)$. If not, then $\text{lca}(u) \neq \text{lca}(v)$ so $u \neq v$. Otherwise $\text{lca}(u) = \text{lca}(v)$ and $u = v$ if and only if $\varphi(u) = \varphi(v)$. All the tests described in this paragraph are implemented with

$$M'(\mu(K), \kappa(K, u), \mu(v)) := \begin{cases} 1 & \text{if } \kappa(v) = \kappa(u) \\ 0 & \text{otherwise.} \end{cases} \quad \square$$

3.4 Graphs with Short Tree-Decompositions

Lemma 15. *Let $\mathcal{G}$ be a monotone class of graphs. Let $k, h : \mathbb{N} \rightarrow \mathbb{R}^+$ be functions with $k, h \in \mathcal{O}(\log n)$. Let $\mathcal{G}'$ be the class of graphs that contains, for each $n \in \mathbb{N}$, every n -vertex graph that has a tree-decomposition $\mathcal{T} := (T, (B_x, x \in V(T)))$ of adhesion-width at most $k(n)$, where $\text{height}(T) \leq h(n)$ and each torso of $\mathcal{T}$ belongs to $\mathcal{G}$. Let $g_i : \mathbb{N} \rightarrow \mathbb{R}$ be a non-decreasing function with $g_i \in \mathcal{O}(\log n)$ for each $i \in [3]$. Suppose that $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme. Then $\mathcal{G}'$ admits a weighted (g'_1, g'_2, g'_3) mixed labelling scheme, where*

$$\begin{aligned} g'_1(n) &= g_1(n) + k(n) \cdot g_2(n) + h(n) \cdot g_3(n) + \mathcal{O}((1 + h(n) + k(n)) \cdot \log \log n) , \\ g'_2(n) &= g_2(n) + \mathcal{O}(1) , \\ g'_3(n) &= (1 + h(n)) \cdot g_3(n) + \mathcal{O}((1 + h(n)) \cdot \log \log n) . \end{aligned}$$

Proof. Let (A, M) be the pair of adjacency and identity testers for the weighted (g_1, g_2, g_3) mixed labelling scheme of $\mathcal{G}$.

As in previous proofs, we show that $\mathcal{G}'$ admits a weak weighted (g'_1, g'_2, g'_3) mixed labelling scheme, which is enough by [Lemma 10](#). Let G^+ be an n -vertex graph in $\mathcal{G}'$, let G be a spanning subgraph of G^+ , let $\mathcal{K}$ be a set of cliques in G^+ , and let $\omega : V(G) \rightarrow \mathbb{N}^+$ be a nice weight function. Thus G^+ has a tree-decomposition $\mathcal{T} := (T, (B_x, x \in V(T)))$ of adhesion-width $k \leq k(n)$, where $h := \text{height}(T) \leq h(n)$ and each torso of $\mathcal{T}$ belongs to $\mathcal{G}$. Let r be the root of T , let $A_r := \emptyset$ and, for each $y \in V(T) \setminus \{r\}$, let $A_y := B_y \cap B_x$, where x is the T -parent of y . The proof is by induction on h , but we require a slightly more detailed inductive hypothesis: We will produce a labelling $\mu : V(G) \rightarrow \{0, 1\}^*$ such that, for each $y \in V(T)$ and each $w \in B_y \setminus A_y$,

$$\begin{aligned} |\mu(w)| &\leq \log \omega(G^+) - \log \omega(w) + g_1(n) \\ &\quad + |A_y| \cdot (g_2(n) + \mathcal{O}(\log \log n)) \\ &\quad + h \cdot g_3(n) + \mathcal{O}((1 + h) \log \log n) . \end{aligned} \tag{15}$$

This satisfies the conditions of the lemma, since $|A_y| \leq k(n)$ for each $y \in V(T)$. Observe that $\omega(T) = \omega(G^+)$, since the sets in $\{B_x \setminus A_x : x \in V(T)\}$ partition $V(G^+)$.

The root labelling μ_r : For each clique K in $G^+\langle B_r \rangle$, let Y_K be the set of T -children y of r such that $B_r \cap B_y = K$. Let $\mathcal{K}_r$ be the set of cliques K in $G^+\langle B_r \rangle$ such that Y_K is non-empty. For each clique $K \in \mathcal{K}_r$ define $\psi(K) := \sum_{y \in Y_K} \omega(T_y)$. Observe that $\sum_{K \in \mathcal{K}_r} \psi(K) = \sum_{y \in L_2(T)} \omega(T_y) = \omega(G) - \omega(B_r)$. For each $v \in B_r$, let $\mathcal{K}_r[v] := \{K \in \mathcal{K}_r : v \in K\}$ and let $\gamma : B_r \rightarrow \mathbb{N}^+$ be the weight function defined by

$$\gamma(v) := k \cdot \omega(v) + \sum_{K \in \mathcal{K}_r[v]} \psi(K) .$$

Since $\mathcal{T}$ has adhesion-width at most k , each clique in $\mathcal{K}_r$ has at most k vertices, so

$$\gamma(B_r) = \sum_{v \in B_r} \gamma(v) \leq k \cdot \omega(B_r) + k \sum_{K \in \mathcal{K}_r} \psi(K) = k \cdot \omega_{\mathcal{T}}(T) = k \cdot \omega(G^+) .$$

Let $\mathcal{K}_0 := \{K \in \mathcal{K} : K \subseteq B_r\}$. Note that the sets $\mathcal{K}_0$ and $\mathcal{K}_r$ have very different roles. Each clique $K \in \mathcal{K}_0$ is an “input” clique and must be assigned a label $\mu(K)$. Each clique in $\mathcal{K}_r$ is defined by an adhesion $B_r \cap B_y$ for at least one T -child y of r . By the definition of $\mathcal{G}'$ and $\mathcal{G}$, there exists a mixed labelling $\mu_r : B_r \rightarrow \{0, 1\}^*$ of $(G^+\langle B_r \rangle, G[B_r], \mathcal{K}_0 \cup \mathcal{K}_r, \gamma)$ for (A, M) . Since $\gamma(B_r) \leq k \cdot \omega(G^+)$ and $\gamma(v) \geq k \cdot \omega(v)$, we have

$$|\mu_r(v)| \leq \log \gamma(B_r) - \log \gamma(v) + g_1(n) \leq \log \omega(G^+) - \log \omega(v) + g_1(n)$$

for each $v \in B_r$.

We make use of two different bounds on $|\mu_r(K)|$. For bounding $g'_1(n)$, we use the following bound for $K \in \mathcal{K}_r$,

$$\begin{aligned} |\mu_r(K)| &\leq \log \gamma(B_r) - \log \min_{v \in K} (\gamma(v)) + g_3(n) \\ &\leq \log \omega(G^+) - \log \psi(K) + g_3(n) + \log k . \end{aligned}$$

For bounding $g'_3(n)$, we use the following bound for $K \in \mathcal{K}_0$,

$$\begin{aligned} |\mu_r(K)| &\leq \log \gamma(B_r) - \log \min_{v \in K} (\gamma(v)) + g_3(n) \\ &\leq \log \omega(G^+) - \log \min_{v \in K} (\omega(v)) + g_3(n) . \end{aligned}$$

The vertex label $\mu(v)$ for $v \in B_r$: For each $v \in B_r$, define $\mu(v) := \mu_r(v)$. Note that the length of $\mu(v)$ satisfies (15), even if $h = 0$.

The clique label $\mu(K)$ for $K \in \mathcal{K}_0$: For each $K \in \mathcal{K}_0$, define $\mu(K) := \mu_r(K)$. Note that the length of $\mu(K)$ satisfies the requirement on $g'_3(n)$.

The local identifier $\kappa(K, v)$ for $K \in \mathcal{K}_0$: For each $K \in \mathcal{K}_0$ and each $v \in K$, the mixed labelling defines a local identifier $\kappa_r(K, v)$ for v in K . For each $K \in \mathcal{K}_0$ and each $v \in K$, define $\kappa(K, v) := \kappa_r(K, v)$. The length of $|\kappa(K, v)| \leq g_2(n)$ obviously satisfies the requirement on $g'_2(n)$.

The subtree labelling μ_y : If $h = 0$, then $V(T) = \{r\}$ and this defines the entire mixed labelling $\mu : V(G^+) \cup \mathcal{K} \rightarrow \{0, 1\}^*$. Otherwise, r has one or more children. For each $z \in V(T) \setminus \{r\}$, let $C_z := B_z \setminus B_r$. We now describe the subproblems on which we apply induction: Let $K \in \mathcal{K}_r$. For each $y \in Y_K$, define

- $G_y^+ := \bigcup_{z \in V(T_y)} G^+(B_z) - B_r$ and $G_y := G[V(G_y^+)]$;
- $\mathcal{T}_y := (T_y, (C_z : z \in V(T_y)))$;
- $\mathcal{K}_y^+ := \{K \in \mathcal{K} : K \cap V(G_y^+) \neq \emptyset\}$ and $\mathcal{K}_y := \{K - B_r : K \in \mathcal{K}_y^+\}$; and
- let $\omega_y : V(G_y^+) \rightarrow \mathbb{N}^+$ be the restriction of $\omega : V(G^+) \rightarrow \mathbb{N}^+$ to the vertices of G_y^+ .

For each $y \in Y_K$, $\mathcal{T}_y$ is a tree-decomposition of G_y^+ and $\text{height}(T_y) \leq h - 1$. By the inductive hypothesis, there exists a weighted $(g'_1(n), g'_2(n), g'_3(n))$ mixed labelling $\mu_y : V(G_y^+) \cup \mathcal{K}_y \rightarrow \{0, 1\}^*$ of G_y^+ , G_y , and $\mathcal{K}_y$ using ω_y that also satisfies (15) for each $w \in V(G_y)$. That is, for each $z \in V(T_y)$ and each $w \in C_z \setminus A_z$,

$$\begin{aligned} |\mu_y(w)| &\leq \log \omega(G_y^+) - \log \omega(w) + g_1(n) \\ &\quad + |A_z \setminus B_r| \cdot (g_2(n) + \mathcal{O}(\log \log n)) \\ &\quad + (h - 1) \cdot g_3(n) + \mathcal{O}((h - 1) \log \log n) . \end{aligned}$$

The subproblem labelling μ_K : Recall that $K \in \mathcal{K}_r$. Define $G_K^+ := \bigcup_{y \in Y_K} G_y^+$, $G_K := \bigcup_{y \in Y_K} G_y$, let $\mathcal{K}_K := \bigcup_{y \in Y_K} \mathcal{K}_y$, and let $\omega_K : V(G_K^+) \rightarrow \mathbb{N}^+$ be the restriction of $\omega : V(G^+) \rightarrow \mathbb{N}^+$ to the vertices in G_K^+ . For each $K \in \mathcal{K}_r$, G_K^+ is the union of pairwise vertex-disjoint graphs $(G_y^+ : y \in Y_K)$. By Lemma 12, there exists a mixed labelling $\mu_K : V(G_K^+) \rightarrow \{0, 1\}^*$ such that, for each $y \in Y_K$, each $z \in V(T_y)$ and each $w \in C_z \setminus A_z$,

$$\begin{aligned} |\mu_K(w)| &\leq \log \omega(G_K^+) - \log \omega(w) + g_1(n) \\ &\quad + |A_z \setminus B_r| \cdot (g_2(n) + \mathcal{O}(\log \log n)) \\ &\quad + (h - 1) \cdot g_3(n) + \mathcal{O}(h \log \log n) . \end{aligned}$$

The vertex label $\mu(v)$ for $v \notin B_r$: Since $\psi(K) = \omega(G_K^+)$,

$$\begin{aligned} |(\mu_r(K), \mu_K(w))| &\leq \log \omega(G^+) - \log \min_{v \in K} (\omega(v)) + g_3(n) + |\mu_K(w)| + \mathcal{O}(\log \log n + \log k) \\ &\leq \log \omega(G^+) - \log \psi(K) + g_3(n) + |\mu_K(w)| + \mathcal{O}(\log \log n) \\ &= \log \omega(G^+) - \log \omega(G_K^+) + g_3(n) + |\mu_K(w)| + \mathcal{O}(\log \log n) \\ &\leq \log \omega(G^+) - \log \omega(w) + g_1(n) \\ &\quad + |A_z \setminus B_r| \cdot (g_2(n) + \mathcal{O}(\log \log n)) \\ &\quad + h \cdot g_3(n) + \mathcal{O}((1 + h) \log \log n) . \end{aligned}$$

For each $v \in A_z \cap B_r$, there is a clique $K \in \mathcal{K}_r$ with $K = B_r \cap B_y \supseteq A_z \cap B_r$ where y is a T -ancestor of z and a T -child of r . Therefore there exists a local identifier $\kappa_r(K, v)$ for v in K

of length at most $g_2(n)$. For each $K \in \mathcal{K}_r$, each $y \in Y_K$, each $z \in V(T_y)$, and each $w \in C_z \setminus A_z$, let

$$\alpha(w) := (\kappa_r(K, v) : v \in A_z \cap B_r \cap N_G(w))$$

and assign the label

$$\mu(w) := (\mu_r(K), \mu_K(w), \alpha(w))$$

The third part of $\mu(w)$ has length at most $|A_z \cap B_r| \cdot (g_2(n) + \mathcal{O}(\log \log n))$, so the length of $\mu(w)$ satisfies (15).

Adjacency Testing: Given the labels $\mu(v)$ and $\mu(w)$ for two vertices v and w of G , the adjacency tester proceeds as follows:

- If v and w are both in B_r , then the adjacency tester uses $\mu_r(v)$ and $\mu_r(w)$ to test if $vw \in E(G[B_r])$.
- If exactly one of v or w is in B_r then, assume without loss of generality that $v \in B_r$ and $w \in G_K$ for some $K \in \mathcal{K}_r$. In this case, the adjacency tester uses $\alpha(w)$ and $\mu_r(K)$ (from $\mu(w)$), and $\mu(v) = \mu_r(v)$ to test if $v = v'$ for some $v' \in N_G(w) \cap B_r$.
- Otherwise, $v \in V(G_K)$ and $w \in V(G_{K'})$ for some $K, K' \in \mathcal{K}_r$. If $\mu_r(K) \neq \mu_r(K')$ then $K \neq K'$, and K separates v and w , so $vw \notin E(G)$. If $K = K'$, then the adjacency tester uses $\mu_K(v)$ and $\mu_K(w)$ to test if $vw \in E(G_K)$.

The clique label $\mu(K)$ for $K \not\subseteq G\langle B_r \rangle$: What remains is to complete the labelling of the cliques in $\mathcal{K}$ and describe the local identifiers. At this point, the only unlabelled cliques in $\mathcal{K}$ are those that contain vertices not in B_r . For each such clique K , there is exactly one child y of r such that G_y^+ contains at least one vertex of K . The clique $K - B_r$ is contained in $\mathcal{K}_y$. There is exactly one clique $K' \in \mathcal{K}_r$ such that $y \in Y_{K'}$. The clique $K - B_r$ is contained in $\mathcal{K}_{K'}$. We define $\mu(K) := (\mu_r(K'), \mu_{K'}(K - B_r))$. Then

$$\begin{aligned} |\mu(K)| &= |\mu_r(K')| + |\mu_{K'}(K - B_r)| + \mathcal{O}(\log \log n) \\ &\leq \log \omega(G^+) - \log \psi(K') + g_3(n) + |\mu_{K'}(K \setminus B_r)| + \mathcal{O}(\log \log n) \\ &\leq \log \omega(G^+) - \log \omega(G_{K'}^+) + g_3(n) + |\mu_{K'}(K \setminus B_r)| + \mathcal{O}(\log \log n) \\ &\leq \log \omega(G^+) - \log \min_{y \in K \setminus B_r} (\omega(y)) + (1+h)g_3(n) + \mathcal{O}((1+h)\log \log n) \\ &\leq \log \omega(G^+) - \log \min_{y \in K} (\omega(y)) + (1+h)g_3(n) + \mathcal{O}((1+h)\log \log n) , \end{aligned}$$

which satisfies the requirement for $g'_3(n)$.

The local identifier $\kappa(K, v)$: Let $K \in \mathcal{K}$. If $K \not\subseteq B_r$ then there exists exactly one $K' \in \mathcal{K}_r$ such that K contains at least one vertex of $G_{K'}$. In this case, $K - B_r \in \mathcal{K}_{K'}$ and there exists a local identifier $\kappa_{K'}(K - B_r, v)$ for each $v \in K \setminus B_r$. For each $v \in K$ define

$$\kappa(K, v) := \begin{cases} (0, \kappa_{K'}(K - B_r, v)) & \text{if } v \notin B_r \\ (1, \kappa_r(K, v)) & \text{if } v \in B_r \text{ and } K \subseteq B_r \\ (2, \kappa_r(K', v)) & \text{if } v \in B_r \text{ and } K \not\subseteq B_r \end{cases}$$

In each case, $|\kappa(K, v)| \leq g_2(n) + \mathcal{O}(1)$.

Identity testing: Given $\mu(K)$, $\mu(v')$ and $\kappa(K, v)$ for some $K \in \mathcal{K}$, $v' \in V(G^+)$ and $v \in K$, the identity tester first checks if $v' \in B_r$.

- If $v' \in B_r$ then $\mu(v') = \mu_r(v')$ and the tester checks if $K \subseteq B_r$.
 - If $K \subseteq B_r$ then $\mu(K) = \mu_r(K)$, $\kappa_r(K, v)$ (from $\kappa(K, v)$) and $\mu(v') = \mu_r(v')$ are used to test if $v' = v$.
 - If $K \not\subseteq B_r$ then there is exactly one T -child y of r such that K contains at least one vertex of G_y^+ and exactly one clique $K' \in \mathcal{K}_r$ such that $y \in Y_{K'}$. In this case v' can only equal v if $v \in K'$. By the definition of $\kappa(K, v)$, $v \in K'$ if and only if the first component of $\kappa(K, v)$ is 2. In this case, $\kappa(K, v)$ contains $\kappa_r(K', v)$. Furthermore, $\mu(K)$ contains $\mu_r(K')$. Thus, $\mu_r(K')$ (from $\mu(K)$), $\mu(v') = \mu_r(v')$, and $\kappa_r(K', v)$ (from $\kappa(K, v)$) can be used to test if $v' = v$.
- If $v' \notin B_r$, then $\mu(v')$ contains $\mu_r(K')$, where K' is the unique clique in $\mathcal{K}_r$ such that $v' \in V(G_{K'}^+)$. The tester then ensures that $v \notin B_r$ by checking that the first component of $\kappa(K, v)$ is 0. If so, then K contains at least one vertex not in B_r , so the first part of $\mu(K)$ is $\mu_r(K'')$, where K'' is the unique clique in $\mathcal{K}_r$ such that $G_{K''}^+$ contains at least one vertex of K . In this case v' can only equal v if $K' = K''$, which can be verified by checking if $\mu_r(K')$ (from $\mu(v')$) and $\mu_r(K'')$ (from $\mu(K)$) are identical. If this is the case, then $v' \in V(G_{K'}^+)$ and $K - B_r \in \mathcal{K}_{K'}$. In this case $\mu_{K'}(K - B_r)$ (from $\mu(K)$), $\mu_{K'}(v')$ (from $\mu(v')$), and $\kappa_{K'}(K - B_r, v)$ (from $\kappa(K, v)$) can be used to test if $v' = v$. $\square$

3.5 Finalizing the Proof of Theorem 3

The following lemma proves Theorem 3, with quantitative bounds.

Lemma 16. *Let $k, g_1, g_2, g_3 : \mathbb{N} \rightarrow \mathbb{N}$ be non-decreasing functions in $\mathcal{O}(\log n)$, let $\mathcal{G}$ be a monotone graph class that admits a weighted (g_1, g_2, g_3) mixed labelling scheme and let $\mathcal{G}'$ be the class of graphs that contains, for each $n \in \mathbb{N}^+$, every n -vertex graph that has a tree-decomposition of adhesion-width at most $k(n)$ in which each torso is a member of $\mathcal{G}$. Then $\mathcal{G}'$ admits a weighted (g'_1, g'_2, g'_3) mixed labelling scheme, where*

$$\begin{aligned}
 g'_1(n) &= g_1(n) + k(n) \cdot g_2(n) + \mathcal{O}\left(\sqrt{k(n) \cdot g_3(n) \cdot \log n}\right) \\
 &\quad + \mathcal{O}\left(\sqrt{\frac{k(n) \log n}{g_3(n)}} (\log \log n)\right) \\
 g'_2(n) &= g_2(n) + \mathcal{O}\left(\sqrt{\frac{g_3(n) \cdot \log n}{k(n)}}\right) + \mathcal{O}(\log \log n) \\
 g'_3(n) &= \mathcal{O}\left(\sqrt{k(n) \cdot g_3(n) \cdot \log n}\right) + \mathcal{O}\left(\sqrt{\frac{k(n) \log n}{g_3(n)}} (\log \log n)\right)
 \end{aligned}$$

Although the expression in Lemma 16 looks intimidating, we will apply it with $k(n) \in \mathcal{O}(1)$ to obtain an $f(n)$ -bit adjacency labelling scheme where $f(n) = \log n + g'_1(n) = \log n +$

$g_1(n) + \mathcal{O}(g_2(n)) + \mathcal{O}(\sqrt{\log n}(\sqrt{g_3(n)} + \log \log n))$. This bound is $(1+o(1))\log n$ for any $g_1, g_2, g_3 \in o(\log n)$.

Proof of Lemma 16. Let G be an n -vertex graph in $\mathcal{G}'$. Then G has a tree-decomposition $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ of adhesion-width at most $k(n)$ and such that $G\langle B_x \rangle \in \mathcal{G}$ for each $x \in V(T)$. Let

$$b(n) := 2^{\sqrt{(g_3(n) \cdot \log n)/k(n)}} \text{ and } h(n) := \log_{b(n)} n = \sqrt{\frac{k(n) \cdot \log n}{g_3(n)}},$$

which implies that $g_3(n) \cdot h(n) = \sqrt{k(n) \cdot g_3(n) \cdot \log n} = k(n) \cdot \log b(n)$. Apply Corollary 9 to get a tree-decomposition $\mathcal{T}' := (T', (B'_x : x \in V(T')))$ of G such that $\text{height}(T') \leq \log_{b(n)} n$, having adhesion-width at most $k(n)$, and such that each torso of $\mathcal{T}'$ has a $b(n)$ -skinny tree-decomposition whose torsos are in $\mathcal{G}$.

By Lemma 14, the torsos of $\mathcal{T}'$ have a weighted $(g'_1(n), g'_2(n), g'_3(n))$ mixed labelling scheme, where

$$\begin{aligned} g'_1(n) &= g_1(n) + \mathcal{O}(k(n)(\log(b(n)) + \log \log n)) \\ &= g_1(n) + \mathcal{O}(\sqrt{k(n) \cdot g_3(n) \cdot \log n}) + \mathcal{O}(k(n) \log \log n) \\ g'_2(n) &= \max\{g_2(n) + \mathcal{O}(1), \mathcal{O}(\log(k(n) \cdot b(n)) + \log \log n)\} \\ &\leq g_2(n) + \mathcal{O}\left(\sqrt{\frac{g_3(n) \cdot \log n}{k(n)}}\right) + \mathcal{O}(\log \log n) \\ g'_3(n) &= g_3(n) + \mathcal{O}(\log \log n) \end{aligned}$$

By Lemma 15, $\mathcal{G}'$ has a weighted $(g''_1(n), g''_2(n), g''_3(n))$ mixed labelling scheme, where

$$\begin{aligned} g''_1(n) &= g'_1(n) + k(n) \cdot g'_2(n) + h(n) \cdot g'_3(n) + \mathcal{O}((1 + h(n)) \log \log n) \\ &= g_1(n) + k(n) \cdot g_2(n) + \mathcal{O}(\sqrt{k(n) \cdot g_3(n) \cdot \log n}) \\ &\quad + \mathcal{O}((1 + k(n) + h(n)) \log \log n) \\ g''_2(n) &= g'_2(n) + \mathcal{O}(1) \\ &= g_2(n) + \mathcal{O}\left(\sqrt{\frac{g_3(n) \cdot \log n}{k(n)}}\right) + \mathcal{O}(\log \log n) \\ g''_3(n) &= (1 + h(n)) \cdot g'_3(n) + \mathcal{O}((1 + h(n)) \log \log n) \\ &= \mathcal{O}(\sqrt{k(n) \cdot g_3(n) \cdot \log n}) + g_3(n) + \mathcal{O}((1 + h(n)) \log \log n) \\ &= \mathcal{O}(\sqrt{k(n) \cdot g_3(n) \cdot \log n}) + \mathcal{O}((1 + h(n)) \log \log n) \end{aligned}$$

Substituting $h(n) = \sqrt{k(n) \log n / g_3(n)}$ into the error terms yields the bounds in the lemma statement. $\square$

4 Proof of Theorem 1

The *strong product* of graphs A and B , denoted by $A \boxtimes B$, is the graph with vertex-set $V(A) \times V(B)$, where distinct vertices $(v, x), (w, y) \in V(A) \times V(B)$ are adjacent if $v = w$ and $xy \in E(B)$, or $x = y$ and $vw \in E(A)$, or $vw \in E(A)$ and $xy \in E(B)$. For an integer $k \geq 0$, let $\mathcal{G}_k$ be the class of graphs isomorphic to a subgraph of $H \boxtimes P$ for some graph H with treewidth at most k and for some path P . This definition includes various classes of interest. For example, Dujmović et al. [9] showed that $\mathcal{G}_8$ includes all planar graphs and, more generally, that a minor-closed class $\mathcal{G}$ is contained in $\mathcal{G}_c$, for some constant c , if and only if some apex graph is not in $\mathcal{G}$. For integers $k, a \geq 0$, let $\mathcal{G}_{k,a}$ be the class of graphs that are isomorphic to a subgraph of $(H \boxtimes P) \oplus K_a$ for some graph H with treewidth at most k and for some path P . Let $\mathcal{G}_{k,a}^*$ be the class of graphs that have a tree-decomposition in which each torso is in $\mathcal{G}_{k,a}$.

A key ingredient in our proof is the following variant of the Robertson–Seymour Graph Minor Structure Theorem, due to Dujmović et al. [9].

Theorem 17 (Graph Minor Product Structure Theorem [9]). *For every proper minor-closed class $\mathcal{G}$ there exists integers $k, a \geq 0$ such that $\mathcal{G} \subseteq \mathcal{G}_{k,a}^*$.*

Note that Dujmović et al. [9] employed Theorem 17 to reprove the result of DeVos et al. [6] mentioned in the introduction, which Gavioille and Labourel [12] used to construct a $(2 + o(1)) \log n$ -bit adjacency labelling scheme for any proper minor-closed class.

Also note that the constant a in Theorem 17 is surprisingly small. Suppose that some graph X is not in $\mathcal{G}$, where $X - A$ is planar for some non-empty set $A \subseteq V(X)$. Then Theorem 42 in [9] (which employs the structure theorem of Dvořák and Thomas [10]) shows that Theorem 17 holds with $a = |A| - 1$. For example, if K_t is not in $\mathcal{G}$, then Theorem 17 holds with $a = \max\{t - 5, 0\}$. Moreover, using the recent polynomial bounds in the Graph Minor Structure Theorem by Gorsky, Seweryn, and Wiederrecht [16], in Theorem 17 one can obtain polynomial bounds on k and a (as a function of the excluded minor). However, it is open whether one can simultaneously get the above optimal bound on a and a polynomial bound on k . Gorsky et al. [16, Conjecture 18.9] conjecture this is possible (also see the discussion in [17]).

Our next tool is a generalization of the results by Dujmović et al. [8] to the weighted mixed labelling setting:

Theorem 18. *For each fixed integer $k \geq 0$, the graph class $\mathcal{G}_k$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme, where $g_1(n), g_3(n) \in \mathcal{O}(\sqrt{\log n})$, and $g_2(n) \in \mathcal{O}(\log \log n)$.*

Theorem 18 is essentially a weighted extension of the $(1 + o(1)) \log n$ -bit adjacency labelling scheme of Dujmović et al. [8] for $\mathcal{G}_k$ and the B-tree variant of this scheme given by Gawrychowski and Janczewski [13].⁴ To obtain this generalization, we employ the standard trick of simulating weight using multiplicity that appears in the proof of Lemma 6.

⁴The labelling of cliques needed for a mixed labelling is implicit in [8], where the label of a vertex w is essentially the label of a clique K_w in a supergraph of G . The cliques $\{K_v : v \in V(G)\}$ are chosen so that, for every edge vw of G , $v, w \in V(K_w)$ or $v, w \in V(K_v)$. See Appendix A for more details.

Since this requires no ideas not already in [8, 13] we defer this discussion to [Appendix A](#), the bulk of which is a review of [8, 13]. Note that the dependency on k in [Theorem 18](#) is not large; in fact $g_1(n), g_3(n) \in \mathcal{O}(\sqrt{\log n} + k \log(k \log n))$, and $g_2(n) \in \mathcal{O}(\log k)$.⁵

[Lemma 13](#) and [Theorem 18](#) immediately imply the following:

Theorem 19. *For fixed integers $a, k \geq 0$, the graph class $\mathcal{G}_{k,a}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme, where $g_1(n), g_3(n) \in \mathcal{O}(\sqrt{\log n})$ and $g_2(n) \in \mathcal{O}(\log \log n)$.*

We can now prove [Theorem 1](#).

Proof of Theorem 1. Let $\mathcal{G}$ be a proper minor-closed class of graphs. By [Theorem 17](#), $\mathcal{G} \subseteq \mathcal{G}_{k,a}^*$ for some k and a that depend only on $\mathcal{G}$. So each graph G in $\mathcal{G}$ has a tree-decomposition $\mathcal{T}$ in which each torso belongs to $\mathcal{G}_{k,a}$. Every graph in $\mathcal{G}_{k,a}$ has no clique with more than $2k + a + 2$ vertices. So $\mathcal{T}$ has adhesion-width at most $2k + a + 2$. By [Theorem 19](#) and [Lemma 16](#), $\mathcal{G}$ has an $f(n)$ -bit adjacency labelling scheme where $f(n) \leq \log n + \mathcal{O}((\log n)^{3/4}) = (1 + o(1)) \log n$. $\square$

5 Conclusion

We conclude by explaining why the proof of [Theorem 1](#) is constructive and that there is a polynomial time encoder. The only difficult part of this process is finding the decomposition described by the Graph Minor Product Structure Theorem ([Theorem 17](#)). Dujmović et al. [9] show how to obtain this decomposition in polynomial-time, given the structural decomposition in the original Robertson-Seymour Graph Minor Structure Theorem. Finding the Robertson-Seymour decomposition is complicated, but polynomial time algorithms exist [5, 16, 19, 20]. Together, these give the parameters k and a , the tree-decomposition $\mathcal{T} := (T, (B_x : x \in V(T)))$ of G and the embedding of each torso of $\mathcal{T}$ into a graph in $\mathcal{G}_{k,a}$. The adjacency labelling scheme in [8] and its modification to weighted mixed labellings in [Appendix A](#) are easily implemented in polynomial time just from their definitions. Thus, for any torso of $\mathcal{T}$, the mixed labelling of any subgraph of the torso and any set of cliques in the torso can be computed in polynomial time. The partition of T into skinny subtrees and the corresponding contracted tree T' and tree-decomposition $\mathcal{T}'$ are easily obtained (even in linear time). The labelling of each torso of $\mathcal{T}'$ given in [Lemma 14](#) requires only computing prefix-free codes ([Lemma 5](#)) and alphabetic codes ([Lemma 6](#) and [Corollary 7](#)) as well as weighted mixed labellings on subgraphs of torso of $\mathcal{T}$. The final vertex label $\mu(v)$ for a vertex of G is then obtained by following the path from the root of T' to the home node of v in $\mathcal{T}'$ and concatenating a sequence of clique labels, the vertex label for v in its home torso, and at most k local identifiers.

⁵The precise expression (after adjusting for notation changes) for the label lengths appears in [8, Section 5.4] as $\log n + \mathcal{O}(s \log \log n + s^{-1} \log n + k \log(k \log n))$. Using the B-tree variant in [13] changes this to $\log n + \mathcal{O}(s + s^{-1} \log n + k \log(k \log n))$. Here s is a free parameter (called k in [8] and called $\log a$ in [13]) that is optimized by choosing $s = \sqrt{\log n}$. This gives the bound on $g_1(n)$ and $g_3(n)$. The bound on $g_2(n)$ is given by $|\gamma(\varphi(u_i))| \in \mathcal{O}(\log k)$ in [8, Section 5.4].

References

- [1] Stephen Alstrup, Søren Dahlgaard, and Mathias Bæk Tejs Knudsen. Optimal induced universal graphs and adjacency labeling for trees. *J. ACM*, 64(4):27:1–27:22, 2017. doi:10.1145/3088513. Preliminary version appeared in FOCS 2015, pages 1311–1326.
- [2] Stephen Alstrup and Theis Rauhe. Improved labeling scheme for ancestor queries. In David Eppstein, editor, *Proc. 13th Annual ACM-SIAM Symposium on Discrete Algorithms* (SODA '02), pages 947–953. ACM/SIAM, 2002.
- [3] Marthe Bonamy, Cyril Gavoille, and Michał Pilipczuk. Shorter labeling schemes for planar graphs. *SIAM J. Discrete Math.*, 36(3):2082–2099, 2022. doi:10.1137/20M1330464. Preliminary version appeared in SODA 2020, pages 446–462.
- [4] Fan R. K. Chung. Universal graphs and induced-universal graphs. *J. Graph Theory*, 14(4):443–454, 1990. doi:10.1002/jgt.3190140408.
- [5] Erik D. Demaine, MohammadTaghi Hajiaghayi, and Ken ichi Kawarabayashi. Algorithmic graph minor theory: Decomposition, approximation, and coloring. In *Proc. 46th Annual IEEE Symposium on Foundations of Computer Science* (FOCS'05), pages 637–646. IEEE Computer Society, 2005. doi:10.1109/SFCS.2005.14.
- [6] Matt DeVos, Guoli Ding, Bogdan Oporowski, Daniel P. Sanders, Bruce Reed, Paul Seymour, and Dirk Vertigan. Excluding any graph as a minor allows a low tree-width 2-coloring. *J. Combinatorial Theory, Series B*, 91(1):25–41, 2004. doi:https://doi.org/10.1016/j.jctb.2003.09.001.
- [7] Reinhard Diestel. *Graph Theory, Fifth Edition*, volume 173 of *Graduate texts in mathematics*. Springer, 2017. doi:10.1007/978-3-662-53622-3.
- [8] Vida Dujmović, Louis Esperet, Cyril Gavoille, Gwenaël Joret, Piotr Micek, and Pat Morin. Adjacency labelling for planar graphs (and beyond). *J. ACM*, 68(6):42:1–42:33, 2021. doi:10.1145/3477542. Preliminary version appeared in FOCS 2020, pages 577–588.
- [9] Vida Dujmović, Gwenaël Joret, Piotr Micek, Pat Morin, Torsten Ueckerdt, and David R. Wood. Planar graphs have bounded queue-number. *J. ACM*, 67(4):22:1–22:38, 2020. doi:10.1145/3385731. Preliminary version appeared in FOCS 2019, pages 862–875.
- [10] Zdeněk Dvořák and Robin Thomas. List-coloring apex-minor-free graphs. *arXiv:1401.1399*, 2014. doi:10.48550/1401.1399.
- [11] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Trans. Inf. Theory*, 21(2):194–203, 1975. doi:10.1109/TIT.1975.1055349.
- [12] Cyril Gavoille and Arnaud Labourel. Shorter implicit representation for planar graphs and bounded treewidth graphs. In Lars Arge, Michael Hoffmann, and Emo Welzl, editors, *Proc. 15th Annual European Symposium on Algorithms* (ESA 2007), volume 4698 of *Lecture Notes in Comput. Sci.*, pages 582–593. Springer, 2007. doi:10.1007/978-3-540-75520-3_52.
- [13] Pawel Gawrychowski and Wojciech Janczewski. Simpler adjacency labeling for planar graphs with B-trees. In Karl Bringmann and Timothy M. Chan, editors, *Proc.*

-
- 1064 5th Symposium on Simplicity in Algorithms (SOSA@SODA 2022), pages 24–36. SIAM,
1065 2022. doi:10.1137/1.9781611977066.3.
- 1066 [14] Edgar N. Gilbert and Edward F. Moore. Variable-length binary encodings. *Bell System*
1067 *Technical Journal*, 38(4):933–967, 1959. doi:10.1002/j.1538-7305.1959.tb01583.
1068 x.
- 1069 [15] Paul C. Gilmore and Alan J. Hoffman. A characterization of comparability graphs
1070 and of interval graphs. *Canadian J. Math.*, 16:539–548, 1964. doi:10.4153/
1071 CJM-1964-055-5.
- 1072 [16] Maximilian Gorsky, Michał T. Seweryn, and Sebastian Wiederrecht. Polynomial
1073 bounds for the graph minor structure theorem. In *66th Proc. IEEE Annual Sympos-*
1074 *ium on Foundations of Computer Science, FOCS 2025*, pages 1961–1978. IEEE, 2025.
1075 doi:10.1109/FOCS63196.2025.00104.
- 1076 [17] Maximilian Gorsky, Michał T. Seweryn, and Sebastian Wiederrecht. The price of
1077 homogeneity is polynomial. *arXiv:2602.01882*, 2026. doi:10.48550/arXiv.2602.
1078 01882.
- 1079 [18] Sampath Kannan, Moni Naor, and Steven Rudich. Implicit representation of graphs.
1080 *SIAM J. Discrete Math.*, 5(4):596–603, 1992. doi:10.1137/0405049. Preliminary ver-
1081 sion appeared in STOC 1988, pages 334–343.
- 1082 [19] Ken-ichi Kawarabayashi, Robin Thomas, and Paul Wollan. A new proof of the flat
1083 wall theorem. *J. Combinatorial Theory, Series B*, 129:158–203, 2018. doi:10.1016/j.
1084 jctb.2017.09.002.
- 1085 [20] Ken-ichi Kawarabayashi, Robin Thomas, and Paul Wollan. Quickly excluding a non-
1086 planar graph. *arXiv:2010.12397*, 2020. doi:10.48550/arXiv.2010.12397.
- 1087 [21] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*.
1088 Addison-Wesley, 2nd edition, 1998.
- 1089 [22] Leon G. Kraft. *A Device for Quantizing, Grouping, and Coding Amplitude Modulated*
1090 *Pulses*. Ms thesis, Massachusetts Institute of Technology, Cambridge, MA, 1949.
1091 Hdl:1721.1/12390.
- 1092 [23] Kurt Mehlhorn. Nearly optimal binary search trees. *Acta Informatica*, 5(4):287–295,
1093 1975. doi:10.1007/BF00264563.
- 1094 [24] John H. Muller. *Local Structure in Graph Classes*. Ph.D. thesis, School of Information
1095 and Computer Science, Georgia Institute of Technology, 1988.
- 1096 [25] C. St. J. A. Nash-Williams. Edge-disjoint spanning trees of finite graphs. *J. London*
1097 *Math. Society*, 36(1):445–450, 1961. doi:10.1112/jlms/s1-36.1.445.
- 1098 [26] Claude E. Shannon. A mathematical theory of communication. *The Bell System Tech-*
1099 *nical Journal*, 27(3):379–423, 1948. doi:10.1002/j.1538-7305.1948.tb01338.x.
- 1100 [27] Claude E. Shannon. A mathematical theory of communication. *The Bell System Tech-*
1101 *nical Journal*, 27(4):623–656, 1948. doi:10.1002/j.1538-7305.1948.tb00917.x.
- 1102 [28] Jeremy P. Spinrad. *Efficient graph representations*, volume 19 of *Fields Institute mono-*
1103 *graphs*. American Mathematical Society, 2003.
-

A Theorem 18

In this appendix, we review the labelling scheme of Dujmović et al. [8] and its improvement by Gawrychowski and Janczewski [13] in order to show the existence of the weighted mixed labellings promised in Theorem 18. Let G be an n -vertex subgraph of $H \boxtimes P$, where H is a graph of treewidth k and P is a path. Without loss of generality, assume that $P = (1, 2, \dots, h)$ and H is an edge-maximal graph of treewidth k . This implies that H has an acyclic orientation in which the out-neighbourhood $N_H^+(v)$ is a clique in H , for each $v \in V(H)$. It follows that the out-degree $|N_H^+(v)|$ is at most k , for each vertex $v \in V(H)$.

Mapping $V(H)$ onto intervals: The first step in [8] is an idea used by Gavaille and Labourel [12] in their labelling scheme for bounded-treewidth graphs. Each vertex v of H is mapped to a real interval $f(v)$. This mapping has the following properties (for some fixed $c > 0$):

1. For any two vertices $v, w \in V(H)$, $f(v) \subseteq f(w)$, $f(w) \subseteq f(v)$ or $f(v) \cap f(w) = \emptyset$.
2. For each $vw \in E(H)$, $f(v) \cap f(w) \neq \emptyset$.
3. For each $x \in \mathbb{R}$, $|\{v \in V(H) : x \in f(v)\}| \leq ck \log n$.

The existence of such a mapping f is easily established by a recursive procedure that takes an open interval (a, b) and a subgraph H' of H . The procedure identifies a set S of at most $k + 1$ vertices in H' such that the components of $H' - S$ can be partitioned into two graphs H'_1 and H'_2 , each with at most $\frac{2}{3}|V(H')|$ vertices. For each $v \in S$, $f(v) := (a, b)$. The procedure is then applied inductively to map the vertices of H'_1 onto subintervals of $(a, (a + b)/2)$ and then again to map the vertices of H'_2 onto subintervals of $((a + b)/2, b)$.

Let I be the interval intersection graph with vertex-set $V(I) := V(H)$ and edge-set $E(I) := \{vw \in \binom{V(H)}{2} : f(v) \cap f(w) \neq \emptyset\}$. Property 2 implies that H is a subgraph of I . Property 3 implies that H has no clique larger than $ck \log n$. The graph I has a proper colouring $\varphi : V(H) \rightarrow \{1, \dots, \lfloor ck \log n \rfloor\}$ [15]. The label for each vertex (v, i) of G will include the $\mathcal{O}(\log \log(kn))$ -bit integer $\varphi(v)$.

Mapping $V(H)$ onto points: For each vertex v of H , let $x_f(v)$ be the midpoint of the interval $f(v)$.⁶ Property 1 then implies:

4. For any clique K in I , there exists $w \in K$ such that $x_f(w) \in f(v)$ for each $v \in K$.

Indeed, a vertex w of K that minimizes the length of $f(w)$ satisfies this condition.

Treating G as a sequence of graphs: Each vertex of G is a pair (v, i) with an *H-coordinate* $v \in V(H)$ and a *P-coordinate* $i \in V(P)$. We may assume, without loss of generality, that G contains at least one vertex with P -coordinate i , for each $i \in \{1, \dots, h\}$. The next idea in [8] is to consider a sequence of subgraphs $R_1, \dots, R_h \subseteq H \boxtimes P$, where $R_i := (H \boxtimes P)[\{(v, j) \in V(G) : j \in [i - 1, i, i + 1]\}]$. Every edge and vertex of G is represented in at least one of these

⁶Any real number $x_f(v)$ in the interior of the interval $f(v)$ would also work.

subgraphs, so $\bigcup_{i=1}^h R_i \supseteq G$. Each vertex of G appears in at most three of these subgraphs, so $\sum_{i=1}^h |V(R_i)| \leq 3n$.

The label of any vertex (v, i) of G has two main parts, whose lengths sum to $\log n + o(\log n)$. The first part, $\lambda_1((v, i))$ is determined by the P -coordinate, i . The second part, $\lambda_2((v, i))$, is obtained from a labelling of R_i , so it represents the H -coordinate, v .

For each $i \in \{1, \dots, h\}$, let $S_i := \{x_f(v) : (v, j) \in V(R_i)\}$. Note that S_i is a set of real numbers, for each $i \in \{1, \dots, h\}$. Before proceeding, [8] defines a sequence of supersets $S_1^+, \dots, S_h^+$. The exact definition of $S_1^+, \dots, S_h^+$ is not necessary for the current discussion. The relevant properties here are:

- (a) $S_i^+ \supseteq S_i$ for each $i \in \{1, \dots, h\}$;
- (b) there is a constant $c > 0$ such that $\sum_{i=1}^h |S_i^+| \leq cn$.

We remark that the contents of the sets $S_1^+, \dots, S_h^+$ are entirely determined by the contents of the sets $S_1, \dots, S_h$. (Formally, the sequence $S_1^+, \dots, S_h^+$ is the result of applying a function $q : (2^{\mathbb{R}})^* \rightarrow (2^{\mathbb{R}})^*$ whose input is a sequence of sets of real numbers and whose output is a sequence of sets of real numbers. The function q is universal; it does not depend on H , P , or G .)

With the sets $S_1^+, \dots, S_h^+$ described, we can now discuss λ_1 . For each vertex (v, i) of G , $\lambda_1((v, i))$ has length $\log n - \log |S_i^+| + \mathcal{O}(\log \log n)$. This part is obtained by applying [Corollary 7](#) to the set $S := \{1, \dots, h\}$ using the weight function $i \mapsto |S_i^+|$ to obtain a prefix-free code $\rho : \{1, \dots, h\} \rightarrow \{0, 1\}^*$. By Property (b), $|\rho(i)| \leq \log n - \log |S_i^+| + \mathcal{O}(1)$ for each $i \in \{1, \dots, h\}$. For each $i \in \{2, \dots, h\}$, a string $\mu(i)$ of length $\mathcal{O}(\log \log n)$ is computed so that $\rho(i-1)$ can be derived from $\rho(i)$ and $\mu(i)$. Then

$$\lambda_1((v, i)) := (\rho(i), \mu(i))$$

for each vertex (v, i) of G .

Bulk trees: In [8], each set S_i^+ is stored in a binary search tree T_i of height $\log |S_i^+| + o(\log n)$. As in [Corollary 7](#), each node x of T_i is assigned a **signature** $\sigma_i(x)$ that describes the root-to- x path $P_{T_i}(x)$ as a binary string of length $\text{depth}_{T_i}(x)$. Note that, for any $x \in V(T_i)$:

$$|\sigma_i(x)| \leq \text{height}(T_i) \leq \log |S_i^+| + o(\log n)$$

For each $i \in \{1, \dots, h\}$, each vertex $(v, j) \in V(R_i)$ is mapped to the node $x_{T_i}(v) \in V(T_i)$ of minimum T_i -depth such that (the real number) $x_{T_i}(v)$ is contained in the (real interval) $f(v)$. The node $x_{T_i}(v)$ always exists because $x_f(v) \in S_i \subseteq S_i^+$ and $x_f(v) \in f(v)$. For each $(v, j) \in V(R_i)$, this defines the **signature** $\sigma((v, i)) := \sigma_i(x_{T_i}(v))$. Note that, for two vertices (v, i) and (v', i') of G , $(v, i) = (v', i')$ if and only if

$$(i, \sigma((v, i)), \varphi(v)) = (i', \sigma((v', i')), \varphi(v')) .$$

For each $i \in \{1, \dots, h\}$ and each $(v, i) \in (V(H) \times \{i\}) \cap V(G)$, define the set

$$X_{v,i} := \{(v, i)\} \cup \left(N_H^+(v) \times \{i-1, i, i+1\} \right) \cap V(G) .$$

Note that, for every edge $(v,i)(w,j)$ of G , $(w,j) \in X_{v,i}$ (if $w \in \{v\} \cup N_H^+(v)$) or $(v,i) \in X_{w,j}$ (if $v \in \{w\} \cup N_H^+(w)$). For each $(w,j) \in X_{v,i}$, we have $x_f(w) \in S_i \subseteq S_i^+ = V(T_i)$. Therefore, $x_{T_i}(w)$ and $\sigma((w,i))$ are well-defined, even when $j \neq i$ and $(w,i) \notin V(G)$.

For each $(v,i) \in (V(H) \times \{i\}) \cap V(G)$, the *extended signature* $\sigma^+((v,i))$ is the signature in $\Sigma_{v,i} := \{\sigma(w,i) : (w,j) \in X_{v,i}\}$ of maximum length. By property 4, every signature in $\Sigma_{v,i}$ is a prefix of $\sigma^+((v,i))$ for each vertex (v,i) of G . The labelling scheme in [8] uses this to implicitly store the signature $\sigma((w,i))$ for each $(w,j) \in X_{v,i}$ inside the label of (v,i) . Specifically, for each $(w,j) \in X_{v,i}$, the label for (v,i) contains an additional $\mathcal{O}(k \log \log(kn))$ bits to store

$$\pi((v,i)) := (|\sigma((w,i))|, \varphi(w), j-i, b(w,j)) \quad : \quad (w,j) \in X_{v,i} \quad . \quad (16)$$

where $b(w,j)$ is a single bit that indicates, for each $(w,j) \in X_{v,i}$ whether or not $(v,i)(w,j) \in E(G)$.

Adjacency testing: Let (v,i) and (w,j) be any two vertices of G , and suppose we are given $\lambda_1((v,i))$, $\sigma^+((v,i))$, $\pi((v,i))$, $\lambda_1((w,j))$, $\sigma^+((w,i))$, and $\pi((w,j))$. (The preceding sentence does not contain a typo! The occurrences of i and j are deliberate.) Given only these values, it is straightforward to test if $(v,i)(w,j)$ is an edge of G . Quickly: Using $\lambda_1((v,i))$ and $\lambda_1((w,j))$, the tester checks if $|j-i| \leq 1$. If not, then (v,i) and (w,j) are not adjacent in G . Otherwise, assume without loss of generality that $j \in \{i-1, i\}$. Then the tester uses $\sigma^+((v,i))$, $\pi((v,i))$ and $\sigma^+((w,i))$, and $\pi((w,j))$ to check if $(w,j) \in X_{v,i}$ or $(v,i) \in X_{w,j}$. If neither of these is the case then (v,i) and (w,j) are not adjacent in G . If $(w,j) \in X_{v,i}$ then the last component of $\pi((v,i))$ is used to test if $(v,i)(w,i) \in E(G)$. If $(v,i) \in X_{w,j}$ then the last component of $\pi((w,i))$ is used to test if $(v,i)(w,i) \in E(G)$.

The main challenge with this approach becomes apparent when we consider an edge of G with endpoints (v,i) and $(w,i-1)$. The label for (v,i) contains all the information the tester needs about (v,i) , but the label for $(w,i-1)$ does not contain $\sigma^+((w,i))$. Instead, it contains $\sigma^+((w,i-1))$. The extended signature $\sigma^+((w,i-1))$ describes a path $P_{T_{i-1}}(x_{T_{i-1}}(w))$ in T_{i-1} , not a path in T_i . To determine if (v,i) and $(w,i-1)$ are adjacent in R_i (and therefore (v,i) and $(w,i-1)$ are potentially adjacent in G) the tester needs the extended signature $\sigma^+((w,i))$ that describes $P_{T_i}(x_{T_i}(w))$. To resolve this problem, the authors of [8] design the binary search trees $T_1, \dots, T_h$ (and the sets $S_1^+, \dots, S_h^+$) so that $\sigma^+((w,i))$ can be derived from $\sigma^+((w,i-1))$ and a *transition code* $\tau((w,i-1))$ of length $o(\log n)$. The transition code $\tau((w,i-1))$ is stored as part of $(w,i-1)$'s label. This is the last piece needed for the second part of the label for a vertex (v,i) of G :

$$\lambda_2((v,i)) := \sigma^+((v,i)), \pi((v,i)), \tau((v,i))$$

In this way, the two main parts of the label for a vertex (v,i) of G have length

$$|\lambda_1((v,i))| + |\lambda_2((v,i))| \leq |\rho(i)| + |\sigma^+((v,i))| + o(\log n) = \log n + o(\log n) \quad .$$

B-trees: In [13], the authors use weight-balanced B-trees to replace the (binary) bulk trees used in [8]. A *B-tree* stores a set of real numbers in the leaves of a (not usually binary) tree B in which all leaves have the same depth. *Weight-balanced B-trees* are parameterized

by a parameter $a \geq 6$. A weight-balanced B-tree B is *semi-balanced* if, for each node x of B , the number of leaves in the subtree B_x is at most $6a^{\text{height}(B_x)}$ and for each *non-root* node x of B , the number of leaves in B_x is at least $\frac{1}{2}a^{\text{height}(B_x)}$. Let B be a weight-balanced B-tree with $r \in \{1, \dots, n\}$ leaves. The weight-balance conditions guarantee that $\text{height}(B) \leq \log_a r + 1$. The weight-balance conditions also ensure that the number of children of any node x of B is at most $12a$. This implies that any root-to-leaf path in a B-tree can be encoded by a sequence of $\log_a r + 1$ integers, each requiring $\lceil \log(12a) \rceil \leq \log a + 5$ bits to represent, for a total of at most $\lceil \log(12a) \rceil \cdot (\log_a r + 1) = \log r + \mathcal{O}(\log_a r + \log a)$ bits. The value of a used in [13] is approximately $2^{\sqrt{\log n}}$, so that any root-to-leaf path in B can be represented using $\log r + \mathcal{O}(\sqrt{\log n})$ bits.

The sequence of binary search trees $T_1, \dots, T_h$ used in [13] is replaced with a sequence of B-trees $B_1, \dots, B_h$. For each $i \in \{1, \dots, h\}$, the leaves of B_i store the values in S_i^+ . For each $i \in \{1, \dots, h\}$, each vertex $(v, j) \in V(R_i)$ is mapped to the lowest-common- B_i -ancestor $x_{B_i}(v)$ of $f(v) \cap S_i^+$. Then, for a vertex (v, i) of G , the signature $\sigma((v, i))$ is the encoding of $P_{B_i}(x_{B_i}(v))$. Then

$$|\sigma((v, i))| \leq \lceil \log(12a) \rceil \cdot (\text{depth}_{B_i}(x_{B_i}(v))) = \lceil \log(12a) \rceil (\text{depth}_{B_i}(\text{lca}_{B_i}(f(v) \cap S_i^+))) .$$

An upper bound on $|\sigma((v, i))|$ can be obtained by using the $\log_a |S_i^+| + 1$ upper bound on $\text{height}(B_i)$:

$$|\sigma((v, i))| \leq \lceil \log(12a) \rceil \text{height}(B_i) \leq \lceil \log(12a) \rceil (\log_a |S_i^+| + 1) = \log |S_i^+| + \mathcal{O}(\sqrt{\log n})$$

The main advantage of using B-trees over the bulk trees in [8] is that in B-trees the rebalancing operations needed to maintain the height bound result in a much simpler transition code $\tau((w, i - 1))$. This simplicity ultimately comes from the fact that, for any $p \in S_{i-1} \cap S_i$, the root-to- p path $P_{B_{i-1}}(p)$ in B_{i-1} and the root-to- p path $P_{B_i}(p)$ in B_i differ by at most a single vertex. More precisely, a subpath xyz in $P_{B_{i-1}}(p)$ can be replaced by a subpath $xy'z$ to give $P_{B_i}(x)$.⁷ In terms of the signature function σ , this means that the difference between these two paths can be described by an $\mathcal{O}(\log \log_a n)$ -bit integer that gives the index of the vertex y being replaced and two $\lceil \log(12a) \rceil$ bit integers that describe where y' is in the list of x 's children and where z is in the list of y' 's children.⁸ As a result of this, the transition code $\tau((w, i - 1))$ used for the B-trees in [13] is shorter (it has length $\mathcal{O}(\sqrt{\log n})$ rather than the $\mathcal{O}(\sqrt{\log n \log \log n})$ required by bulk trees) and, on a computer with $\mathcal{O}(\log n)$ -bit words, the translation from $\sigma^+((w, i - 1))$ to $\sigma^+((w, i))$ can be done in constant time, resulting in a constant-time adjacency testing procedure. (All of the other adjacency-testing operations for both schemes can be done in constant-time.)

Except for the new definition of the basic signature σ and the transition function τ , every other aspect of the resulting labelling scheme and adjacency testing procedure is as described in [8].

⁷There are two exceptions. Occasionally, $P_{B_i}(x)$ is obtained by removing the first vertex of $P_{B_{i-1}}(x)$ or by adding a vertex at the beginning of $P_{B_{i-1}}(x)$. Since these are distracting and easily handled, we ignore them here.

⁸Determining $\sigma^+((w, i))$ from $\sigma^+((w, i - 1))$ requires a bit more work, since it is possible that $x_{B_i}(w) \neq x_{B_{i-1}}(w)$. This is common to both labelling schemes and does not affect the upcoming discussion. The important thing for what comes next is the definition of $x_{B_i}(w) = \text{lca}_{B_i}(f(w) \cap S_i^+)$.

A weighted-labelling scheme: We now describe the changes to [8, 13] needed to prove **Theorem 18**. **Theorem 18** involves two graphs G^+ and G with $G \subseteq G^+ \subseteq H \boxtimes P$, a set $\mathcal{K}$ of cliques in G^+ , and a weight function $\omega : V(G^+) \rightarrow \mathbb{N}^+$. First, observe that the final verification performed by the adjacency tester before confirming the presence of an edge vw checks a bit, specific to the edge vw , that is explicitly stored in $\lambda(v)$ or $\lambda(w)$ to indicate the presence (or not) of vw in $E(G)$. This bit serves no other purpose. Thus, the adjacency labelling for the supergraph G^+ of G can be made into an adjacency labelling for G by changing a subset of the bits in each label. From this point on we assume that $G = G^+$.

We proceed exactly as in the proof of **Lemma 6**. For each $i \in \{1, \dots, h\}$ and each $v \in (V(H) \times \{i\}) \cap V(G)$, define

$$S_{\omega,i}[v] := \{(x_f(v), 1), \dots, (x_f(v), \omega((v, i)))\}$$

and define $S_{\omega,i} := \bigcup_{(v,j) \in V(R_i)} S_{\omega,i}[v]$. For each $i \in \{1, \dots, h\}$, treat $S_{\omega,i}$ as a totally ordered set where order is determined by lexicographic comparison. Note that $|S_{\omega,i}| = \omega(G[V(H) \times \{i-1, i, i+1\}])$ for each $i \in \{1, \dots, h\}$ and $\sum_{i=1}^h |S_{\omega,i}| \leq 3\omega(G)$.

We use the convention that a real interval (a, b) contains $(x, j) \in S_i^+$ if (the real interval) (a, b) contains (the real number) x . In particular, for a vertex $v \in V(H)$ and $i \in \{1, \dots, h\}$, $f(v) \cap S_i^+ := \{(x, j) \in S_i^+ : x \in f(v)\}$. Then, just as in the unweighted case $x_{B_i}(v) = \text{lca}_{B_i}(f(v) \cap S_i^+)$. The only difference at this point is that we apply the machinery in [8, 13] beginning with the sets $S_{\omega,1}, \dots, S_{\omega,h}$. This results in sets $S_{\omega,1}^+, \dots, S_{\omega,h}^+$ of total size $\sum_{i=1}^h |S_{\omega,i}^+| \in \mathcal{O}(\omega(G))$. This has the desired effect because a B-tree of height h has at most $6a^h$ leaves. The node $\text{lca}_{B_i}(S_{\omega,i}[v])$ is the root of a subtree of B_i with at least $|S_{\omega,i}[v]| = \omega((v, i))$ leaves. Therefore,

$$\begin{aligned} \text{depth}_{B_i}(x_{B_i}(v)) &= \text{depth}_{B_i}(\text{lca}_{B_i}(f(v) \cap S_{\omega,i})) \\ &\leq \text{depth}_{B_i}(\text{lca}_{B_i}(S_{\omega,i}[v])) \\ &\leq \text{height}(B_i) - \log_a \omega((v, i)) + 1 \\ &\leq \log_a |S_{\omega,i}^+| - \log_a \omega((v, i)) + 2. \end{aligned}$$

Then

$$\begin{aligned} |\lambda_2((v, i))| &\leq \lceil \log(12a) \rceil \text{depth}_{B_i}(x_{B_i}(v)) \\ &\leq \log |S_{\omega,i}^+| - \log \omega((v, i)) + \mathcal{O}(\log_a |S_{\omega,i}^+| + \log a) \\ &\leq \log |S_{\omega,i}^+| - \log \omega((v, i)) + \mathcal{O}(\sqrt{\log n}) \end{aligned}$$

and

$$\begin{aligned} |\lambda_1((v, i))| + |\lambda_2((v, i))| &\leq \log \omega(G) - \log |S_{\omega,i}^+| + \mathcal{O}(\log \log n) \\ &\quad + \log |S_{\omega,i}^+| - \log \omega((v, i)) + \mathcal{O}(\sqrt{\log n}) \\ &= \log \omega(G) - \log \omega((v, i)) + \mathcal{O}(\sqrt{\log n}). \end{aligned}$$

Clique labelling and identity testing: In the preceding, we have argued that the lengths of the adjacency labels promised in **Theorem 18** are achievable. All that remains is to describe the clique labels, the local identifiers, and the identity testing procedure.

Let K be a clique in $G = G^+$. Then $K \subseteq R_i$ for some $i \in \{1, \dots, h\}$. Let $X_K := \{v : (v, i) \in K\}$. Then $H[X_K]$ is a clique and there exists $v^* \in X_K$ such that $\{v^*\} \cup N_H^+(v^*) \supseteq X_K$. Therefore, there exists $(v^*, i) \in K$ such that $X_{v^*, i} \supseteq K$. Recall that $\sigma^+((v^*, i))$ and $\pi((v^*, i))$ contain enough information to recover $\sigma((w, j))$, $\varphi(w)$, and $j - i$ for each $(w, j) \in X_{v^*, i}$. Order the vertices of $X_{v^*, i}$ as $(w_1, j_1), \dots, (w_r, j_r)$ in the same order that the information about these vertices appears in $\pi((v^*, i))$.

Define the clique label $\lambda(K) := (\lambda((v^*, i)), b(K))$, where $b(K) := b_1(K), \dots, b_{|X_{v^*, i}|}(K)$ is a bit-vector that indicates, for each $p \in \{1, \dots, |X_{v^*, i}|\}$, whether $(w_p, j_p) \in K$. The length of this label is

$$\begin{aligned} |\lambda(K)| &= |\lambda((v^*, i))| + \mathcal{O}(k) \\ &\leq \log \omega(G) - \log \omega((v^*, i)) + \mathcal{O}(\sqrt{\log n}) \\ &\leq \log \omega(G) - \log \min\{\omega((w, j)) : (w, j) \in K\} + \mathcal{O}(\sqrt{\log n}) . \end{aligned}$$

Let $(v_1, \ell_1), \dots, (v_{|K|}, \ell_{|K|})$ be the vertices of K in the order that these vertices appear in the sequence $(w_1, j_1), \dots, (w_r, j_r)$. For each $m \in \{1, \dots, |K|\}$, define the local identifier $\kappa(K, (v_m, \ell_m)) := m$. Then $|\kappa(K, v)| \in \mathcal{O}(\log k)$ for each $K \in \mathcal{K}$ and each $v \in K$.

Given $\lambda(K)$, $\lambda((w', j'))$, and the local identifier $m := \kappa(K, (w, j))$, the identity tester first extracts the bit-vector $b(K)$ from $\lambda(K)$. It then computes the minimum integer p such that $\sum_{q=1}^p b_q(K) = m$. This integer p is the index of the target vertex (w_p, j_p) within the sequence $X_{v^*, i}$. Finally, the tester proceeds exactly like the adjacency testing algorithm to determine if the input vertex (w', j') is equal to (w_p, j_p) . It does this by using $\lambda((w', j'))$ and the structural information about (w_p, j_p) that is inherently stored inside $\lambda((v^*, i))$ (which is provided as the first part of $\lambda(K)$).